

```
1 // ===== FILE: Assets/DressSort/Scripts/Screens/App.cs =====
2 using System.Collections;
3 using System.Collections.Generic;
4 using UnityEngine;
5 using UnityEngine.UI;
6 namespace DressSort
7 {
8     public enum ScreenId
9     {
10         Home,
11         Game,
12         Reward,
13         DressUp,
14         Pack,
15         Workshop,
16     }
17     /// <summary>
18     /// 所有页面都是同一个 Canvas 下的面板，靠显示隐藏切换，没有场景加载，
19     /// 这样在微信小游戏上不会有切页面的卡顿。
20     /// </summary>
21     public abstract class Panel : MonoBehaviour
22     {
23         protected App app;
24         protected RectTransform root;
25         public void Init(App owner, RectTransform frame)
26         {
27             app = owner;
28             root = frame;
29             Build();
30         }
31         protected abstract void Build();
32         public virtual void OnShow() { }
33         public virtual void OnHide() { }
34     }
35     public class App : MonoBehaviour
36     {
37         [SerializeField] GameDatabase database;
38         public GameDatabase Database => database;
39         public WardrobeService Wardrobe { get; private set; }
40         /// <summary>刚通关的那一关，Reward 页要用。</summary>
41         public LevelDef PendingLevel { get; set; }
42         public LevelDef CurrentLevel { get; set; }
43         /// <summary>刚通关那一关掉的材料，奖励页要显示。</summary>
44         public int[] PendingMaterials { get; set; }
45         /// <summary>回到首页时要提示的一句话，首页显示后清空。</summary>
46         public string Notice { get; set; }
47         /// <summary>体力不够回到首页时，弹出看广告恢复体力。</summary>
48         public bool OfferEnergyAd { get; set; }
49         readonly Dictionary<ScreenId, Panel> panels = new Dictionary<ScreenId, Panel>();
50         readonly Dictionary<ScreenId, RectTransform> safeRoots = new Dictionary<ScreenId, RectTra...
```

```
51     ScreenId current = ScreenId.Home;
52     Canvas canvas;
53     Image pageCover;
54     bool firstShow = true;
55     bool booted;
56     void Awake()
57     {
58         if (Application.isPlaying)
59             Boot(null, null);
60     }
61     /// <summary>
62     /// 建界面。传入相机时 Canvas 走 ScreenSpaceCamera,
63     /// 这样编辑器脚本不进播放模式也能把界面渲到 RenderTexture 上截图验证。
64     /// </summary>
65     public void Boot(Camera captureCamera, GameDatabase overrideDatabase)
66     {
67         if (booted) return;
68         if (overrideDatabase != null)
69             database = overrideDatabase;
70         if (database == null)
71         {
72             Debug.LogError("[一裙又一裙] 没有接上 GameDatabase, 先跑菜单里的「重建资源与场景」");
73             enabled = false;
74             return;
75         }
76         booted = true;
77         if (Application.isPlaying)
78         {
79             Sfx.Install();
80             CdnPortraits.Warm();
81         }
82         BindChrome(database);
83         UiKit.Skin = database;
84         Wardrobe = new WardrobeService(database);
85         ClearBuiltUi();
86         BuildCanvas(captureCamera);
87         BuildPanels();
88         BuildPageCover();
89         Show(ScreenId.Home);
90     #if UNITY_EDITOR
91         if (Application.isPlaying && captureCamera == null)
92             StartCoroutine(PlayBootSplash());
93     #endif
94     }
95     public void SetDatabase(GameDatabase value) => database = value;
96     static void BindChrome(GameDatabase db)
97     {
98         if (db == null) return;
99     #if UNITY_EDITOR
100         const string ui = "Assets/DressSort/Art/Ui/";
```

```
101     if (db.uiBtnTeal == null)
102     {
103         db.uiBtnTeal = UnityEditor.AssetDatabase.LoadAssetAtPath<Sprite>(ui + "btn_tea.p...
104         db.uiBtnPink = UnityEditor.AssetDatabase.LoadAssetAtPath<Sprite>(ui + "btn_pink.p...
105         db.uiBtnWhite = UnityEditor.AssetDatabase.LoadAssetAtPath<Sprite>(ui + "btn_white...
106         db.uiCard = UnityEditor.AssetDatabase.LoadAssetAtPath<Sprite>(ui + "card.png");
107         db.uiCardOn = UnityEditor.AssetDatabase.LoadAssetAtPath<Sprite>(ui + "card_on.png");
108         db.uiLane = UnityEditor.AssetDatabase.LoadAssetAtPath<Sprite>(ui + "lane.png");
109     }
110     if (db.uiBgs == null || db.uiBgs.Count == 0 || db.uiBg == null || db.uiBg.name == "bg")
111     {
112         db.uiBgs = new List<Sprite>
113         {
114             LoadUiSprite(ui + "Bgs/bg_room"),
115             LoadUiSprite(ui + "Bgs/bg_boutique"),
116             LoadUiSprite(ui + "Bgs/bg_sky"),
117             LoadUiSprite(ui + "Bgs/bg_garden"),
118             LoadUiSprite(ui + "Bgs/bg_stage"),
119         };
120         db.uiBgIndex = 4;
121         db.uiBg = db.uiBgs[4] != null ? db.uiBgs[4] : LoadUiSprite(ui + "bg");
122     }
123     if (db.pageLoading == null)
124         db.pageLoading = LoadUiSprite("Assets/DressSort/Art/Loading/page_yellow");
125     {
126         const string dressup = ui + "DressUp/";
127         db.dressupTray = LoadUiSprite(dressup + "tray");
128         db.dressupBtnCream = LoadUiSprite(dressup + "btn_cream");
129         db.dressupBtnPeach = LoadUiSprite(dressup + "btn_peach");
130         db.dressupSave = LoadUiSprite(dressup + "btn_save");
131         db.dressupCard = LoadUiSprite(dressup + "card");
132         db.dressupCardOn = LoadUiSprite(dressup + "card_on");
133         db.dressupCardLock = LoadUiSprite(dressup + "card_lock");
134     }
135     {
136         const string home = "Assets/DressSort/Art/Ui/Home/";
137         if (db.btnHomeStart == null)
138         {
139             db.btnHomeStart = LoadUiSprite(home + "btn_home_start");
140             db.btnHomeDress = LoadUiSprite(home + "btn_home_dress");
141             db.iconHomeCircle = LoadUiSprite(home + "icon_home_circle");
142             db.iconHomeRank = LoadUiSprite(home + "icon_home_rank");
143             db.iconHomeCheckin = LoadUiSprite(home + "icon_home_checkin");
144             db.iconHomeWorkshop = LoadUiSprite(home + "icon_home_workshop");
145             db.iconHomeQuest = LoadUiSprite(home + "icon_home_quest");
146             db.iconHomeEvent = LoadUiSprite(home + "icon_home_event");
147         }
148         if (db.iconHomeEnergy == null)
149             db.iconHomeEnergy = LoadUiSprite(home + "icon_home_energy");
150         if (db.labelHomeCircle == null)
```

```
151     {
152         db.labelHomeCircle = LoadUISprite(home + "label_home_circle");
153         db.labelHomeRank = LoadUISprite(home + "label_home_rank");
154         db.labelHomeCheckin = LoadUISprite(home + "label_home_checkin");
155         db.labelHomeWorkshop = LoadUISprite(home + "label_home_workshop");
156         db.labelHomeQuest = LoadUISprite(home + "label_home_quest");
157         db.labelHomeEvent = LoadUISprite(home + "label_home_event");
158     }
159 }
160 if (db.btnGameUndo == null || db.btnGameGear == null)
161 {
162     db.btnGameUndo = LoadUISprite(ui + "Game/btn_undo");
163     db.btnGameShuffle = LoadUISprite(ui + "Game/btn_shuffle");
164     db.btnGameBack = LoadUISprite(ui + "Game/btn_back");
165     db.btnGameSteps = LoadUISprite(ui + "DressUp/tab_track");
166     db.btnGameGear = LoadUISprite(ui + "Game/btn_gear");
167 }
168 if (db.btnGameSwap == null)
169 {
170     db.btnGameSwap = LoadUISprite(ui + "Game/btn_swap");
171     db.gameBadge = LoadUISprite(ui + "Game/badge");
172     db.gameRod = LoadUISprite(ui + "Game/rod");
173     db.gamePauseBoard = LoadUISprite(ui + "Game/pause_board");
174 }
175 if (db.gameParcel == null)
176 {
177     db.gameParcel = LoadUISprite(ui + "Game/parcel");
178     db.gameDustCover = LoadUISprite(ui + "Game/dust_cover");
179     db.gameCoverTag = LoadUISprite(ui + "Game/cover_tag");
180 }
181 if (db.gameLock == null)
182 {
183     db.gameLock = LoadUISprite(ui + "Game/lock");
184     db.gameKey = LoadUISprite(ui + "Game/key");
185     db.gameBubble = LoadUISprite(ui + "Game/bubble");
186 }
187 if (db.gameAlarm == null)
188     db.gameAlarm = LoadUISprite(ui + "Game/alarm");
189 if (db.mysteryItem == null)
190     db.mysteryItem = UnityEditor.AssetDatabase.LoadAssetAtPath<ItemDef>(
191         "Assets/DressSort/Data/Items/mystery.asset");
192 if (db.packCarpet == null)
193     db.packCarpet = Resources.Load<Sprite>("Pack/bg_carpet");
194 if (db.packLane == null)
195 {
196     db.packLane = LoadUISprite(ui + "Pack/lane");
197     db.packLaneOn = LoadUISprite(ui + "Pack/lane_on");
198     db.packLaneReady = LoadUISprite(ui + "Pack/lane_ready");
199     db.packBox = LoadUISprite(ui + "Pack/box");
200     db.packBtn = LoadUISprite(ui + "Pack/btn_pack");
```

```

201         db.packRefill = LoadUiSprite(ui + "Pack/btn_refill");
202         db.packLock = LoadUiSprite(ui + "Pack/lock");
203         db.packAd = LoadUiSprite(ui + "Pack/ad");
204         db.packCheck = LoadUiSprite(ui + "Pack/check");
205     }
206     if (db.boardBgs == null || db.boardBgs.Count < GameDatabase.BoardThemes.Length
207         || db.boardRods == null || db.boardRods.Count < GameDatabase.BoardThemes.Length)
208     {
209         db.boardBgs = new List<Sprite>();
210         db.boardHangers = new List<Sprite>();
211         db.boardRods = new List<Sprite>();
212         foreach (string theme in GameDatabase.BoardThemes)
213         {
214             bool shop = theme == "shop";
215             db.boardBgs.Add(LoadUiSprite(ui + "Bgs/" + (shop ? "bg_shop" : "board_" + the...
216             db.boardHangers.Add(LoadUiSprite(ui + "Game/hanger_" + (shop ? "closet" : the...
217             db.boardRods.Add(LoadUiSprite(ui + "Game/" + (shop ? "rod" : "rod_" + theme)));
218         }
219         db.boardBgBand = 20;
220         if (db.iconHanger == null)
221             db.iconHanger = LoadUiSprite(ui + "Game/hanger_closet");
222     }
223     if (db.craftMats == null || db.craftMats.Count < CraftCatalog.MatCount || db.craftMat...
224     {
225         db.craftMats = new List<Sprite>();
226         foreach (string key in CraftCatalog.MatKeys)
227             db.craftMats.Add(LoadUiSprite(ui + "Workshop/mat_" + key));
228         db.craftCard = LoadUiSprite(ui + "Workshop/card");
229         db.craftBg = LoadUiSprite(ui + "Workshop/bg_workshop");
230     }
231     if (db.rewardGiftClosed == null || db.craftBtnMint == null)
232         BindRewardArt(db, ui);
233     if (db.uiBtnTeal != null || db.uiBg != null)
234         UnityEditor.EditorUtility.SetDirty(db);
235 #endif
236     }
237 #if UNITY_EDITOR
238     public static readonly string[] RewardConfettiNames =
239     {
240         "fx_strip_gold", "fx_strip_red", "fx_strip_mint", "fx_curl_gold", "fx_curl_pink",
241         "fx_curl_blue", "fx_dot", "fx_star", "fx_heart", "fx_square",
242     };
243     public static void BindRewardArt(GameDatabase db, string ui = "Assets/DressSort/Art/Ui/")
244     {
245         string reward = ui + "Reward/";
246         db.craftBtnMint = LoadUiSprite(ui + "Workshop/btn_mint");
247         db.rewardBg = LoadUiSprite(reward + "bg_reward");
248         db.rewardGiftClosed = LoadUiSprite(reward + "gift_closed");
249         db.rewardGiftBase = LoadUiSprite(reward + "gift_base");
250         db.rewardGiftLid = LoadUiSprite(reward + "gift_lid");

```

```

251     db.rewardRays = LoadUiSprite(reward + "rays");
252     db.rewardBtn = LoadUiSprite(reward + "btn_gold");
253     db.rewardRibbon = LoadUiSprite(reward + "ribbon");
254     db.rewardPlate = LoadUiSprite(reward + "plate");
255     db.rewardPopper = LoadUiSprite(reward + "fx_popper");
256     db.rewardSparkle = LoadUiSprite(reward + "fx_sparkle");
257     db.rewardConfetti = new List<Sprite>();
258     foreach (string name in RewardConfettiNames)
259         db.rewardConfetti.Add(LoadUiSprite(reward + name));
260     UnityEditor.EditorUtility.SetDirty(db);
261 }
262 static Sprite LoadUiSprite(string pathNoExt)
263 {
264     var sprite = UnityEditor.AssetDatabase.LoadAssetAtPath<Sprite>(pathNoExt + ".jpg");
265     return sprite != null
266         ? sprite
267         : UnityEditor.AssetDatabase.LoadAssetAtPath<Sprite>(pathNoExt + ".png");
268 }
269 #endif
270 void ClearBuiltUi()
271 {
272     for (int i = transform.childCount - 1; i >= 0; i--)
273     {
274         Transform child = transform.GetChild(i);
275         if (Application.isPlaying)
276             Destroy(child.gameObject);
277         else
278             DestroyImmediate(child.gameObject);
279     }
280     panels.Clear();
281     safeRoots.Clear();
282     canvas = null;
283 }
284 void BuildCanvas(Camera captureCamera)
285 {
286     var go = new GameObject("UI", typeof(Canvas), typeof(CanvasScaler), typeof(GraphicRay...
287     go.transform.SetParent(transform, false);
288     canvas = go.GetComponent<Canvas>();
289     if (captureCamera != null)
290     {
291         canvas.renderMode = RenderMode.ScreenSpaceCamera;
292         canvas.worldCamera = captureCamera;
293         canvas.planeDistance = 10f;
294     }
295     else
296     {
297         canvas.renderMode = RenderMode.ScreenSpaceOverlay;
298     }
299     ScreenFit.Apply(captureCamera != null ? captureCamera : Camera.main, canvas);
300     var es = FindObjectOfType<UnityEngine.EventSystems.EventSystem>();

```

```

301     if (es == null)
302     {
303         var esGo = new GameObject("EventSystem",
304             typeof(UnityEngine.EventSystems.EventSystem),
305             typeof(UnityEngine.EventSystems.StandaloneInputModule));
306         es = esGo.GetComponent<UnityEngine.EventSystems.EventSystem>();
307     }
308     WxBridge.OverrideTouch(es.gameObject);
309     if (WxBridge.IsMiniGame)
310         WxBridge.ShowShareMenu();
311 }
312 void LateUpdate()
313 {
314     if (!booted || canvas == null) return;
315     ScreenFit.Apply(Camera.main, canvas);
316     foreach (var pair in safeRoots)
317         ScreenFit.ApplySafeArea(pair.Value);
318 }
319 void BuildPanels()
320 {
321     Add<HomePanel>(ScreenId.Home);
322     Add<GamePanel>(ScreenId.Game);
323     Add<RewardPanel>(ScreenId.Reward);
324     Add<DressUpPanel>(ScreenId.DressUp);
325     Add<PackPanel>(ScreenId.Pack);
326     Add<WorkshopPanel>(ScreenId.Workshop);
327 }
328 void Add<T>(ScreenId id) where T : Panel
329 {
330     RectTransform frame = UiKit.Stretch(canvas.transform, id.ToString());
331     UiKit.Backdrop(frame, database.ActiveBg);
332     RectTransform safe = UiKit.Stretch(frame, "Safe");
333     ScreenFit.ApplySafeArea(safe);
334     safeRoots[id] = safe;
335     var panel = frame.gameObject.AddComponent<T>();
336     panel.Init(this, safe);
337     frame.gameObject.SetActive(false);
338     panels[id] = panel;
339 }
340 #if UNITY_EDITOR
341 IEnumerator PlayBootSplash()
342 {
343     string path = System.IO.Path.GetFullPath("Assets/DressSort/Art/Loading/loading.mp4");
344     if (!System.IO.File.Exists(path) || canvas == null)
345         yield break;
346     RectTransform frame = UiKit.Stretch(canvas.transform, "BootSplash");
347     frame.SetAsLastSibling();
348     var raw = frame.gameObject.AddComponent<RawImage>();
349     raw.color = Color.white;
350     raw.raycastTarget = true;

```

```
351     var player = frame.gameObject.AddComponent<UnityEngine.Video.VideoPlayer>();
352     player.playOnAwake = false;
353     player.renderMode = UnityEngine.Video.VideoRenderMode.RenderTexture;
354     player.audioOutputMode = UnityEngine.Video.VideoAudioOutputMode.None;
355     player.isLooping = false;
356     player.url = "file://" + path;
357     var rt = new RenderTexture(720, 1280, 0);
358     player.targetTexture = rt;
359     raw.texture = rt;
360     player.Prepare();
361     float wait = 0f;
362     while (!player.isPrepared && wait < 2f)
363     {
364         wait += Time.unscaledDeltaTime;
365         yield return null;
366     }
367     if (player.isPrepared)
368     {
369         player.Play();
370         while (player.isPlaying)
371             yield return null;
372     }
373     player.Stop();
374     if (rt != null)
375         rt.Release();
376     if (Application.isPlaying)
377         Destroy(frame.gameObject);
378     else
379         DestroyImmediate(frame.gameObject);
380 }
381 #endif
382 void BuildPageCover()
383 {
384     if (database.pageLoading == null) return;
385     RectTransform frame = UiKit.Stretch(canvas.transform, "PageLoading");
386     pageCover = frame.gameObject.AddComponent<Image>();
387     pageCover.sprite = database.pageLoading;
388     pageCover.color = Color.white;
389     pageCover.raycastTarget = true;
390     frame.SetAsLastSibling();
391     frame.gameObject.SetActive(false);
392 }
393 public void Show(ScreenId id)
394 {
395     if (firstShow || pageCover == null || !Application.isPlaying)
396     {
397         firstShow = false;
398         ApplyShow(id);
399         return;
400     }
```

```

401     StopCoroutine("ShowCovered");
402     StartCoroutine>ShowCovered(id));
403 }
404 IEnumerator ShowCovered(ScreenId id)
405 {
406     pageCover.gameObject.SetActive(true);
407     pageCover.transform.SetAsLastSibling();
408     yield return null;
409     ApplyShow(id);
410     yield return new WaitForSeconds(0.2f);
411     pageCover.gameObject.SetActive(false);
412 }
413 void ApplyShow(ScreenId id)
414 {
415     if (panels.TryGetValue(current, out Panel previous) && previous != null)
416     {
417         previous.OnHide();
418         previous.gameObject.SetActive(false);
419     }
420     current = id;
421     BgmId bgm = BgmId.Lobby;
422     if (id == ScreenId.Game)
423         bgm = BgmId.Play;
424     else if (id == ScreenId.Pack)
425         bgm = BgmId.Event;
426     else if (id == ScreenId.Reward)
427         bgm = BgmId.None;
428     Sfx.SetBgm(bgm);
429     if (panels.TryGetValue(id, out Panel next) && next != null)
430     {
431         next.gameObject.SetActive(true);
432         next.OnShow();
433     }
434 }
435 public bool StartLevel(LevelDef level)
436 {
437     if (level == null) return false;
438     if (!Wardrobe.SpendEnergy())
439         return false;
440     CurrentLevel = level;
441     Show(ScreenId.Game);
442     return true;
443 }
444 public T PanelOf<T>(ScreenId id) where T : Panel =>
445     panels.TryGetValue(id, out Panel panel) ? panel as T : null;
446 public LevelDef LevelAt(int index) => LevelCatalog.Get(database, index);
447 public int LevelCount => LevelCatalog.Count;
448 /// <summary>每关只玩一次，永远接着打下一关。全部通关返回 null。</summary>
449 public LevelDef NextLevel => LevelAt(Wardrobe.LevelsCleared + 1);
450 /// <summary>切换全页背景。后面做换装背景页时直接调这个。</summary>

```

```

451     public void SetBackground(int index)
452     {
453         database.SelectBg(index);
454         Sprite bg = database.ActiveBg;
455         foreach (var pair in panels)
456         {
457             Transform backdrop = pair.Value.transform.Find("Backdrop");
458             if (backdrop == null) continue;
459             var image = backdrop.GetComponent<Image>();
460             if (image != null)
461                 image.sprite = bg;
462         }
463     }
464 }
465 }
466 // ===== FILE: Assets/DressSort/Scripts/Core/SortBoard.cs =====
467 using System;
468 using System.Collections.Generic;
469 namespace DressSort
470 {
471     /// <summary>
472     /// 棋盘的纯逻辑，不碰任何 Unity 类型。
473     ///
474     /// 每一列用一个 List 表示，索引 0 是视觉上的最上面一件，最后一个是最下面一件。
475     /// 玩家点一列：手里那件插到该列最上面，该列最下面那件顶出来到手里。
476     /// 手里那件落到中间后，若某列只剩一件异款且正好是这列的款，异款会立刻转到列底。
477     ///
478     /// 关卡机制：
479     /// 包裹：发牌时盖住的格子，看不到是哪款，落到列底才拆开。
480     /// 防尘罩：罩住的列不能点，也看不到里面，叠好指定数量的其它列后拉开，之后一直开着。
481     /// 专属列：列顶挂着款式气泡，这列只认这一款。
482     /// 锁和钥匙：锁住的列看得见但不能点；带钥匙的那件被顶出来时，从左往右开一把锁。
483     /// 限时闹钟：挂闹钟的那件要在倒数归零前被顶出来，否则闹钟响了这局失败。
484     /// </summary>
485     public class SortBoard
486     {
487         public readonly struct Settle
488         {
489             public readonly int Column;
490             public readonly int FromIndex;
491             public Settle(int column, int fromIndex)
492             {
493                 Column = column;
494                 FromIndex = fromIndex;
495             }
496         }
497         public const int Empty = -1;
498         /// <summary>格子上的机制标记跟着格子一起移动，所以直接编码在格子的值里。</summary>
499         const int TypeMask = 0xFF;
500         const int WrapBit = 1 << 8;

```

```

501     const int KeyBit = 1 << 9;
502     const int AlarmShift = 10;
503     const int AlarmMask = 0xF << AlarmShift;
504     const int MaxAlarms = 15;
505     /// <summary>只在发牌自检时用：给每格编号，回放时看它第几步被顶出来。</summary>
506     const int TagShift = 16;
507     const int TagMask = 0xFFF << TagShift;
508     /// <summary>一步记录。Row 为 -1 是普通插列，否则是「交换」换走的那一格。</summary>
509     readonly struct Move
510     {
511         public readonly int Column;
512         public readonly int Row;
513         public Move(int column, int row)
514         {
515             Column = column;
516             Row = row;
517         }
518     }
519     readonly List<int>[] columns;
520     readonly int[] coverNeed;
521     readonly bool[] opened;
522     readonly bool[] lockAtStart;
523     readonly bool[] unlocked;
524     readonly int[] target;
525     readonly int[] alarmDeadline = new int[MaxAlarms + 1];
526     readonly Stack<Move> history = new Stack<Move>();
527     readonly Stack<Settle[]> settleHistory = new Stack<Settle[]>();
528     readonly List<int> recipe = new List<int>();
529     int lastEjectedRow;
530     static readonly Settle[] NoSettles = Array.Empty<Settle>();
531     public int ColumnCount { get; }
532     public int ColumnHeight { get; }
533     /// <summary>款式数量。问号那件的编号就等于它，排在所有款式之后。</summary>
534     public int PaletteSize { get; }
535     public int MysteryIndex => PaletteSize;
536     public int Held { get; private set; } = Empty;
537     public int Steps { get; private set; }
538     public int MoveLimit { get; }
539     /// <summary>最近一次手里的裙子落下后，自动换到列底的那些异款。</summary>
540     public Settle[] LastSettles { get; private set; } = NoSettles;
541     /// <summary>最近一步刚拉开防尘罩的列。</summary>
542     public int[] LastOpened { get; private set; } = Array.Empty<int>();
543     /// <summary>最近一步用钥匙打开的列。</summary>
544     public int[] LastUnlocked { get; private set; } = Array.Empty<int>();
545     /// <summary>最近一步顶出来的那件挂着闹钟，也就是关掉了一个。</summary>
546     public bool LastAlarmOff { get; private set; }
547     /// <summary>发牌时倒推的走法正着排一遍。编辑器自检拿它确认能解。</summary>
548     public IReadOnlyList<int> Recipe => recipe;
549     /// <summary>发牌后按倒推走法正着回放过一遍，确认在步数和闹钟限制内能解开。</summary>
550     public bool Verified { get; private set; }

```

```

551     public SortBoard(int columnCount, int columnHeight, int paletteSize, int moveLimit)
552     {
553         ColumnCount = columnCount;
554         ColumnHeight = columnHeight;
555         PaletteSize = paletteSize;
556         MoveLimit = moveLimit;
557         columns = new List<int>[columnCount];
558         for (int i = 0; i < columnCount; i++)
559             columns[i] = new List<int>(columnHeight + 1);
560         coverNeed = new int[columnCount];
561         opened = new bool[columnCount];
562         lockAtStart = new bool[columnCount];
563         unlocked = new bool[columnCount];
564         target = new int[columnCount];
565         for (int i = 0; i < columnCount; i++)
566             target[i] = Empty;
567     }
568     SortBoard(SortBoard src, bool openLocks)
569         : this(src.ColumnCount, src.ColumnHeight, src.PaletteSize, src.MoveLimit)
570     {
571         for (int c = 0; c < ColumnCount; c++)
572         {
573             columns[c].AddRange(src.columns[c]);
574             coverNeed[c] = src.coverNeed[c];
575             opened[c] = src.opened[c];
576             lockAtStart[c] = src.lockAtStart[c];
577             unlocked[c] = openLocks || src.unlocked[c];
578             target[c] = src.target[c];
579         }
580         Array.Copy(src.alarmDeadline, alarmDeadline, alarmDeadline.Length);
581         Held = src.Held;
582     }
583     /// <summary>每列要先叠好几列才拉开防尘罩，0 表示没有罩。发牌前设。</summary>
584     public void SetCovers(ReadOnlyList<int> needs)
585     {
586         for (int c = 0; c < ColumnCount; c++)
587         {
588             coverNeed[c] = needs != null && c < needs.Count ? Math.Max(0, needs[c]) : 0;
589             opened[c] = false;
590         }
591     }
592     /// <summary>大于 0 的列开局上锁。发牌前设。</summary>
593     public void SetLocks(ReadOnlyList<int> flags)
594     {
595         for (int c = 0; c < ColumnCount; c++)
596         {
597             lockAtStart[c] = flags != null && c < flags.Count && flags[c] > 0 && coverNeed[c]...
598             unlocked[c] = !lockAtStart[c];
599         }
600     }

```

```

601    /// <summary>大于 0 的列挂专属气泡。已解状态下第 c 列就是第 c 款，所以气泡款式就是列号。</summary>
602    public void SetTargets(ReadOnlyList<int> flags)
603    {
604        for (int c = 0; c < ColumnCount; c++)
605            target[c] = flags != null && c < flags.Count && flags[c] > 0 ? c : Empty;
606    }
607    /// <summary>只给画面读，返回去掉标记的款式。</summary>
608    public IReadOnlyList<int> Column(int index)
609    {
610        List<int> raw = columns[index];
611        var types = new int[raw.Count];
612        for (int i = 0; i < raw.Count; i++)
613            types[i] = raw[i] & TypeMask;
614        return types;
615    }
616    int Cell(int column, int row)
617    {
618        List<int> list = columns[column];
619        return row >= 0 && row < list.Count ? list[row] : 0;
620    }
621    public bool IsWrapped(int column, int row) => (Cell(column, row) & WrapBit) != 0;
622    public bool HasKey(int column, int row) => (Cell(column, row) & KeyBit) != 0;
623    /// <summary>这格闹钟还剩几步，没有闹钟返回 -1。</summary>
624    public int AlarmLeft(int column, int row)
625    {
626        int alarm = (Cell(column, row) & AlarmMask) >> AlarmShift;
627        return alarm > 0 ? alarmDeadline[alarm] - Steps : -1;
628    }
629    public bool IsCovered(int column) => coverNeed[column] > 0 && !opened[column];
630    public bool HasLock(int column) => lockAtStart[column];
631    public bool IsLocked(int column) => !unlocked[column];
632    /// <summary>专属列认的款式，没有返回 -1。</summary>
633    public int TargetOf(int column) => target[column];
634    public int CoverNeed(int column) => coverNeed[column];
635    /// <summary>防尘罩还差几列才拉开。</summary>
636    public int CoverLeft(int column) =>
637        IsCovered(column) ? Math.Max(0, coverNeed[column] - SolvedCount) : 0;
638    /// <summary>已经叠好的列数，罩着的不算。</summary>
639    public int SolvedCount
640    {
641        get
642        {
643            int count = 0;
644            for (int c = 0; c < ColumnCount; c++)
645            {
646                if (!IsCovered(c) && IsColumnSolved(c)) count++;
647            }
648            return count;
649        }
650    }

```

```
651     ///
```

```

701     int id = 1;
702     while (id < MaxAlarms && alarmDeadline[id] > 0) id++;
703     alarmDeadline[id] = Steps + steps;
704     columns[column][row] = (columns[column][row] & ~AlarmMask) | (id << AlarmShift);
705 }
706 public int CountIn(int index) => columns[index].Count;
707 public bool CanUndo => history.Count > 0;
708 public bool OutOfMoves => Steps >= MoveLimit;
709 void ResetLast()
710 {
711     LastSettles = NoSettles;
712     LastOpened = Array.Empty<int>();
713     LastUnlocked = Array.Empty<int>();
714     LastAlarmOff = false;
715 }
716 // ----- 生成
717 /// <summary>
718 /// 从已解状态倒推生成。倒着走的每一步都是正向走法的逆操作，
719 /// 所以把这串操作正着走一遍就能解开。
720 ///
721 /// 有防尘罩或锁时分两段倒推，正着走就是：先只动没罩没锁的列，
722 /// 叠好够数的列拉开罩子、顶出钥匙开锁，再动其余的列。
723 /// 生成完正着回放一遍，自动沉底让局面走岔、或者闹钟来不及关的，换下一个种子重来。
724 /// 同一个种子永远发出同一盘。
725 /// </summary>
726 public void Deal(int scrambleMoves, int seed, int parcels = 0, int alarms = 0)
727 {
728     Verified = false;
729     for (int attempt = 0; attempt < 80 && !Verified; attempt++)
730     {
731         var random = new Random(unchecked(seed + attempt * 7919));
732         Generate(scrambleMoves, random, parcels);
733         Verified = PlaceKeysAndAlarms(random, alarms);
734     }
735     if (!Verified)
736     {
737         for (int c = 0; c < ColumnCount; c++)
738             unlocked[c] = true;
739     }
740     Steps = 0;
741     history.Clear();
742     settleHistory.Clear();
743     ResetLast();
744 }
745 void Generate(int scrambleMoves, Random random, int parcels)
746 {
747     for (int c = 0; c < ColumnCount; c++)
748     {
749         columns[c].Clear();
750         for (int r = 0; r < ColumnHeight; r++)

```

```
751         columns[c].Add(c);
752         opened[c] = false;
753         unlocked[c] = !lockAtStart[c];
754     }
755     Array.Clear(alarmDeadline, 0, alarmDeadline.Length);
756     Held = MysteryIndex;
757     recipe.Clear();
758     var free = new List<int>();
759     bool anyLock = false;
760     int maxNeed = 0;
761     for (int c = 0; c < ColumnCount; c++)
762     {
763         if (coverNeed[c] > 0)
764             maxNeed = Math.Max(maxNeed, coverNeed[c]);
765         else if (lockAtStart[c])
766             anyLock = true;
767         else
768             free.Add(c);
769     }
770     maxNeed = Math.Min(maxNeed, free.Count);
771     if (maxNeed == 0 && !anyLock)
772     {
773         Scramble(AllColumns(), scrambleMoves, random);
774     }
775     else
776     {
777         Shuffle(free, random);
778         var keep = new HashSet<int>(free.GetRange(0, maxNeed));
779         var late = new List<int>();
780         for (int c = 0; c < ColumnCount; c++)
781         {
782             if (!keep.Contains(c)) late.Add(c);
783         }
784         free.Sort();
785         int lateMoves = scrambleMoves * 2 / 5;
786         Scramble(late, lateMoves, random);
787         Scramble(free, scrambleMoves - lateMoves, random);
788     }
789     recipe.Reverse();
790     WrapRandom(parcel, random);
791     Unwrap();
792 }
793 <summary>
794 /// 按倒推走法正着回放，记下每格第几步被顶出来，
795 /// 钥匙挂在锁住的列第一次被点之前就会顶出来的格子上，闹钟的倒数留出余量。
796 /// 再按真实规则回放一遍确认能解。
797 </summary>
798 bool PlaceKeysAndAlarms(Random random, int alarms)
799 {
800     int locks = 0;
```

```
801     for (int c = 0; c < ColumnCount; c++)
802     {
803         if (lockAtStart[c]) locks++;
804     }
805     alarms = Math.Min(alarms, MaxAlarms);
806     for (int c = 0; c < ColumnCount; c++)
807     {
808         List<int> list = columns[c];
809         for (int r = 0; r < list.Count; r++)
810             list[r] |= (c * 16 + r + 1) << TagShift;
811     }
812     var ejectStep = new Dictionary<int, int>();
813     int firstLockedPlay = int.MaxValue;
814     var trace = new SortBoard(this, true);
815     foreach (int column in recipe)
816     {
817         if (trace.IsWin()) break;
818         if (!trace.CanPlay(column))
819             return Fail();
820         if (lockAtStart[column] && firstLockedPlay == int.MaxValue)
821             firstLockedPlay = trace.Steps + 1;
822         trace.Play(column);
823         int tag = (trace.lastEjectedRaw & TagMask) >> TagShift;
824         if (tag > 0 && !ejectStep.ContainsKey(tag))
825             ejectStep[tag] = trace.Steps;
826     }
827     if (!trace.IsWin() || trace.Steps > MoveLimit)
828         return Fail();
829     if (locks > 0 || alarms > 0)
830     {
831         var keyCells = new List<int>();
832         var alarmCells = new List<int>();
833         for (int c = 0; c < ColumnCount; c++)
834         {
835             if (coverNeed[c] > 0) continue;
836             List<int> list = columns[c];
837             for (int r = 0; r < list.Count; r++)
838             {
839                 if ((list[r] & WrapBit) != 0) continue;
840                 int tag = (list[r] & TagMask) >> TagShift;
841                 if (!ejectStep.TryGetValue(tag, out int step)) continue;
842                 if (step < firstLockedPlay && !lockAtStart[c]) keyCells.Add(c * 1000 + r);
843                 if (step >= 4) alarmCells.Add(c * 1000 + r);
844             }
845         }
846         Shuffle(keyCells, random);
847         if (keyCells.Count < locks)
848             return Fail();
849         var used = new HashSet<int>();
850         for (int i = 0; i < locks; i++)
```

```
851     {
852         int cell = keyCells[i];
853         columns[cell / 1000][cell % 1000] |= KeyBit;
854         used.Add(cell);
855     }
856     Shuffle(alarmCells, random);
857     int placed = 0;
858     for (int i = 0; i < alarmCells.Count && placed < alarms; i++)
859     {
860         int cell = alarmCells[i];
861         if (used.Contains(cell)) continue;
862         List<int> list = columns[cell / 1000];
863         int r = cell % 1000;
864         int step = ejectStep[(list[r] & TagMask) >> TagShift];
865         placed++;
866         alarmDeadline[placed] = Math.Min(MoveLimit, step + Math.Max(6, step * 3 / 5) ...
867         list[r] |= placed << AlarmShift;
868     }
869     if (placed < alarms)
870         return Fail();
871 }
872 StripTags();
873 var check = new SortBoard(this, false);
874 foreach (int column in recipe)
875 {
876     if (check.IsWin()) break;
877     if (!check.CanPlay(column))
878         return Fail();
879     check.Play(column);
880     if (check.AlarmRang)
881         return Fail();
882 }
883 return check.IsWin() || Fail();
884 }
885 bool Fail()
886 {
887     for (int c = 0; c < ColumnCount; c++)
888     {
889         List<int> list = columns[c];
890         for (int r = 0; r < list.Count; r++)
891             list[r] &= TypeMask | WrapBit;
892     }
893     Array.Clear(alarmDeadline, 0, alarmDeadline.Length);
894     return false;
895 }
896 void StripTags()
897 {
898     for (int c = 0; c < ColumnCount; c++)
899     {
900         List<int> list = columns[c];
```

```
901         for (int r = 0; r < list.Count; r++)
902             list[r] &= ~TagMask;
903     }
904 }
905 List<int> AllColumns()
906 {
907     var all = new List<int>(ColumnCount);
908     for (int c = 0; c < ColumnCount; c++)
909         all.Add(c);
910     return all;
911 }
912 void Scramble(List<int> pool, int moves, Random random)
913 {
914     if (pool.Count == 0) return;
915     int previous = -1;
916     for (int k = 0; k < moves; k++)
917     {
918         int c = pool[random.Next(pool.Count)];
919         if (c == previous && pool.Count > 1)
920         {
921             int at = (pool.IndexOf(c) + 1 + random.Next(pool.Count - 1)) % pool.Count;
922             c = pool[at];
923         }
924         previous = c;
925         ReverseMove(c);
926         recipe.Add(c);
927     }
928 }
929 static void Shuffle(List<int> list, Random random)
930 {
931     for (int i = list.Count - 1; i > 0; i--)
932     {
933         int j = random.Next(i + 1);
934         int tmp = list[i];
935         list[i] = list[j];
936         list[j] = tmp;
937     }
938 }
939 /// <summary>包裹只盖没罩的列，列底那格不盖，盖了也会马上拆开。</summary>
940 void WrapRandom(int parcels, Random random)
941 {
942     if (parcels <= 0) return;
943     var cells = new List<int>();
944     for (int c = 0; c < ColumnCount; c++)
945     {
946         if (coverNeed[c] > 0) continue;
947         for (int r = 0; r < columns[c].Count - 1; r++)
948             cells.Add(c * 1000 + r);
949     }
950     Shuffle(cells, random);
```

```

951     int count = Math.Min(parcels, cells.Count);
952     for (int i = 0; i < count; i++)
953         columns[cells[i] / 1000][cells[i] % 1000] |= WrapBit;
954 }
955 // ----- 走子
956 public bool CanPlay(int column) =>
957     column >= 0 && column < ColumnCount
958     && Held != Empty
959     && !OutOfMoves
960     && !IsCovered(column)
961     && !IsLocked(column)
962     && !IsColumnSolved(column)
963     && !AlarmRang;
964 /// <summary>
965 /// 一列只剩一件异款、手里又是这列的款时，把那件异款挪到列底，
966 /// 这样下一步交换会直接把它顶出来。已经在列底则不必挪。
967 /// 还有包裹、罩着或锁着的列不挪，免得提前露底；专属列只为它认的款挪。
968 /// </summary>
969 public bool TryAutoRotate(int column, out int oddIndex)
970 {
971     oddIndex = -1;
972     if (Held == Empty || Held == MysteryIndex) return false;
973     if (column < 0 || column >= ColumnCount) return false;
974     if (IsCovered(column) || IsLocked(column)) return false;
975     if (target[column] != Empty && target[column] != Held) return false;
976     List<int> list = columns[column];
977     if (list.Count < 2) return false;
978     int odd = -1;
979     int matches = 0;
980     for (int i = 0; i < list.Count; i++)
981     {
982         if ((list[i] & WrapBit) != 0) return false;
983         if ((list[i] & TypeMask) == Held)
984         {
985             matches++;
986             continue;
987         }
988         if (odd >= 0) return false;
989         odd = i;
990     }
991     if (odd < 0 || matches != list.Count - 1) return false;
992     if (odd == list.Count - 1) return false;
993     oddIndex = odd;
994     return true;
995 }
996 /// <summary>
997 /// 手里的裙子已经在中间时调用：把所有「只剩一件异款」的列里，
998 /// 那件异款挪到列底。发牌和每次交换后都会走一遍。
999 /// </summary>
1000 public Settle[] SettleReady()

```

```

1001     {
1002         var list = new List<Settle>();
1003         for (int c = 0; c < ColumnCount; c++)
1004         {
1005             if (!TryAutoRotate(c, out int odd)) continue;
1006             RotateToBottom(columns[c], odd);
1007             list.Add(new Settle(c, odd));
1008         }
1009         LastSettles = list.Count == 0 ? NoSettles : list.ToArray();
1010         return LastSettles;
1011     }
1012     /// <summary>手里那件插到列顶，列底那件顶出来。新到手的那件若对上某列，异款立刻沉底。</summary>
1013     public int Play(int column)
1014     {
1015         List<int> list = columns[column];
1016         list.Insert(0, Held);
1017         int last = list.Count - 1;
1018         int raw = list[last];
1019         list.RemoveAt(last);
1020         lastEjectedRaw = raw;
1021         Held = raw & TypeMask;
1022         Steps++;
1023         LastAlarmOff = (raw & AlarmMask) != 0;
1024         LastUnlocked = (raw & KeyBit) != 0 ? UnlockNext() : Array.Empty<int>();
1025         history.Push(new Move(column, -1));
1026         settleHistory.Push(SettleReady());
1027         AfterMove();
1028         return Held;
1029     }
1030     int[] UnlockNext()
1031     {
1032         for (int c = 0; c < ColumnCount; c++)
1033         {
1034             if (unlocked[c]) continue;
1035             unlocked[c] = true;
1036             return new[] { c };
1037         }
1038         return Array.Empty<int>();
1039     }
1040     public bool CanSwap(int column, int row)
1041     {
1042         if (column < 0 || column >= ColumnCount) return false;
1043         if (IsCovered(column) || IsLocked(column)) return false;
1044         if (row < 0 || row >= columns[column].Count || Held == Empty) return false;
1045         int cell = columns[column][row];
1046         if ((cell & (WrapBit | KeyBit | AlarmMask)) != 0) return false;
1047         return (cell & TypeMask) != Held;
1048     }
1049     /// <summary>「交换」道具：手里那件和架上任意一格直接对调，不算步数。</summary>
1050     public int Swap(int column, int row)

```

```

1051     {
1052         List<int> list = columns[column];
1053         int taken = list[row] & TypeMask;
1054         list[row] = Held;
1055         Held = taken;
1056         LastAlarmOff = false;
1057         LastUnlocked = Array.Empty<int>();
1058         history.Push(new Move(column, row));
1059         settleHistory.Push(SettleReady());
1060         AfterMove();
1061         return taken;
1062     }
1063     /// <summary>最近一次撤回的是不是「交换」，面板据此退还道具次数。</summary>
1064     public bool LastUndoWasSwap { get; private set; }
1065     /// <summary>拆开的包裹、拉开的防尘罩、打开的锁、拆掉的闹钟，撤回时都不复原。</summary>
1066     public int Undo()
1067     {
1068         if (history.Count == 0) return Empty;
1069         Move move = history.Pop();
1070         Settle[] settles = settleHistory.Count > 0 ? settleHistory.Pop() : NoSettles;
1071         for (int i = settles.Length - 1; i >= 0; i--)
1072             MoveBottomTo(columns[settles[i].Column], settles[i].FromIndex);
1073         ResetLast();
1074         LastUndoWasSwap = move.Row >= 0;
1075         if (LastUndoWasSwap)
1076         {
1077             List<int> list = columns[move.Column];
1078             int back = list[move.Row] & TypeMask;
1079             list[move.Row] = Held;
1080             Held = back;
1081             Unwrap();
1082             return back;
1083         }
1084         int outgoing = ReverseMove(move.Column);
1085         if (Steps > 0) Steps--;
1086         Unwrap();
1087         return outgoing;
1088     }
1089     void AfterMove()
1090     {
1091         Unwrap();
1092         int solved = SolvedCount;
1093         List<int> now = null;
1094         for (int c = 0; c < ColumnCount; c++)
1095         {
1096             if (!IsCovered(c) || solved < coverNeed[c]) continue;
1097             opened[c] = true;
1098             (now ??= new List<int>()).Add(c);
1099         }
1100         LastOpened = now == null ? Array.Empty<int>() : now.ToArray();

```

```
1101     if (now != null)
1102     {
1103         for (int i = 0; i < now.Count; i++)
1104             Unwrap(now[i]);
1105     }
1106 }
1107 /// <summary>每列最下面那格拆开；整列同款的也全部拆开。</summary>
1108 void Unwrap()
1109 {
1110     for (int c = 0; c < ColumnCount; c++)
1111         Unwrap(c);
1112 }
1113 void Unwrap(int column)
1114 {
1115     List<int> list = columns[column];
1116     if (list.Count == 0) return;
1117     int last = list.Count - 1;
1118     list[last] &= ~WrapBit;
1119     if (!IsColumnSolved(column)) return;
1120     for (int i = 0; i < list.Count; i++)
1121         list[i] &= ~WrapBit;
1122 }
1123 static void RotateToBottom(List<int> list, int index)
1124 {
1125     int item = list[index];
1126     list.RemoveAt(index);
1127     list.Add(item);
1128 }
1129 static void MoveBottomTo(List<int> list, int index)
1130 {
1131     if (list.Count == 0) return;
1132     int item = list[list.Count - 1];
1133     list.RemoveAt(list.Count - 1);
1134     index = MathfClamp(index, 0, list.Count);
1135     list.Insert(index, item);
1136 }
1137 static int MathfClamp(int value, int min, int max)
1138 {
1139     if (value < min) return min;
1140     if (value > max) return max;
1141     return value;
1142 }
1143 int ReverseMove(int column)
1144 {
1145     List<int> list = columns[column];
1146     list.Add(Held);
1147     int outgoing = list[0] & TypeMask;
1148     list.RemoveAt(0);
1149     Held = outgoing;
1150     return outgoing;
```

```
1151     }
1152     // ----- 判定
1153     public bool IsColumnSolved(int column)
1154     {
1155         List<int> list = columns[column];
1156         if (list.Count != ColumnHeight) return false;
1157         int first = list[0] & TypeMask;
1158         if (target[column] != Empty && first != target[column]) return false;
1159         for (int i = 1; i < list.Count; i++)
1160         {
1161             if ((list[i] & TypeMask) != first) return false;
1162         }
1163         return true;
1164     }
1165     /// <summary>
1166     /// 每种款式的数量正好等于列高，所以只要每列都同款，
1167     /// 各列的款式必然互不相同，不用额外查重。
1168     /// </summary>
1169     public bool IsWin()
1170     {
1171         if (Held != MysteryIndex) return false;
1172         for (int c = 0; c < ColumnCount; c++)
1173         {
1174             if (!IsColumnSolved(c)) return false;
1175         }
1176         return true;
1177     }
1178 }
1179 }
1180 // ===== FILE: Assets/DressSort/Scripts/Core/PackBoard.cs =====
1181 using System;
1182 using System.Collections.Generic;
1183 namespace DressSort
1184 {
1185     /// <summary>
1186     /// 活动页「同款装箱」。不碰 Unity。
1187     /// 先点一列，再点另一列：只倒最上面连着的同款。
1188     /// 对面是空列，或者顶上也是同款，就能倒。空位有几件就倒几件，倒满为止，剩下的留在原列。
1189     /// 一列被同款放满就可以装箱。收纳箱满 4 格算一箱，装满 3 箱过关。
1190     /// 这一局裙子种类开局就定了。补充不用等台上清空：有空位的列各叠一件队列里的同款，原来的留在下面。
1191     /// </summary>
1192     public class PackBoard
1193     {
1194         public const int ColumnCount = 6;
1195         public const int ColumnHeight = 8;
1196         public const int StartOpen = 3;
1197         public const int ProgressColumn = 3;
1198         public const int AdColumnA = 4;
1199         public const int AdColumnB = 5;
1200         public const int BoxSlots = 4;
```

```

1201     public const int BoxesToWin = 3;
1202     public struct Cell
1203     {
1204         public int Type;
1205         public bool Revealed;
1206     }
1207     public enum TapResult
1208     {
1209         Ignored,
1210         Moved,
1211         Selected,
1212     }
1213     public struct PackResult
1214     {
1215         public int Column;
1216         public int Type;
1217         public bool Shipped;
1218         public bool Won;
1219     }
1220     readonly List<Cell>[] columns;
1221     readonly List<int> reserve = new List<int>();
1222     readonly bool[] open = new bool[ColumnCount];
1223     public int Selected { get; private set; } = -1;
1224     public int BoxFilled { get; private set; }
1225     public int BoxesDone { get; private set; }
1226     public bool Won { get; private set; }
1227     public int ReserveCount => reserve.Count;
1228     /// <summary>这一盘按记录的走法能装完。</summary>
1229     public bool Proven { get; private set; }
1230     public PackBoard(int paletteSize, int seed) : this()
1231     {
1232         Deal(Math.Max(1, paletteSize), seed);
1233     }
1234     PackBoard()
1235     {
1236         columns = new List<Cell>[ColumnCount];
1237         for (int i = 0; i < ColumnCount; i++)
1238             columns[i] = new List<Cell>(ColumnHeight);
1239         for (int i = 0; i < StartOpen; i++)
1240             open[i] = true;
1241     }
1242     public bool IsOpen(int column) => column >= 0 && column < ColumnCount && open[column];
1243     public bool IsAdColumn(int column) => column == AdColumnA || column == AdColumnB;
1244     public IReadOnlyList<Cell> Column(int index) => columns[index];
1245     public bool UnlockAd(int column)
1246     {
1247         if (!IsAdColumn(column) || open[column]) return false;
1248         open[column] = true;
1249         return true;
1250     }

```

```
1251    /// <summary>最上面连续同款的件数。列表第 0 个就是画面上最上面那件。</summary>
1252    public int GroupSize(int column)
1253    {
1254        if (!IsOpen(column)) return 0;
1255        List<Cell> list = columns[column];
1256        if (list.Count == 0) return 0;
1257        int type = list[0].Type;
1258        int count = 1;
1259        for (int i = 1; i < list.Count; i++)
1260        {
1261            if (list[i].Type != type) break;
1262            count++;
1263        }
1264        return count;
1265    }
1266    /// <summary>顶上同色，空列或者对方顶上也是同款。不管空位够不够。</summary>
1267    public bool TopsMatch(int from, int to)
1268    {
1269        if (!IsOpen(from) || !IsOpen(to) || from == to) return false;
1270        if (GroupSize(from) <= 0) return false;
1271        List<Cell> dest = columns[to];
1272        if (dest.Count == 0) return true;
1273        return dest[0].Type == columns[from][0].Type;
1274    }
1275    /// <summary>这次能倒过去的件数。顶上同色，有空位就倒，最多倒到把空位填满。</summary>
1276    public int PourCount(int from, int to)
1277    {
1278        if (!CanMove(from, to)) return 0;
1279        int space = ColumnHeight - columns[to].Count;
1280        int group = GroupSize(from);
1281        return space < group ? space : group;
1282    }
1283    public bool CanMove(int from, int to)
1284    {
1285        if (!TopsMatch(from, to)) return false;
1286        return columns[to].Count < ColumnHeight;
1287    }
1288    public TapResult Tap(int column)
1289    {
1290        if (Won || !IsOpen(column)) return TapResult.Ignored;
1291        if (Selected < 0 || column == Selected || !CanMove(Selected, column))
1292        {
1293            Selected = column == Selected ? -1 : column;
1294            return TapResult.Selected;
1295        }
1296        Move(Selected, column);
1297        Selected = -1;
1298        return TapResult.Moved;
1299    }
1300    public bool IsFullUniform(int column)
```

```
1301     {
1302         if (!IsOpen(column)) return false;
1303         List<Cell> list = columns[column];
1304         if (list.Count != ColumnHeight) return false;
1305         int type = list[0].Type;
1306         for (int i = 1; i < list.Count; i++)
1307         {
1308             if (list[i].Type != type) return false;
1309         }
1310         return true;
1311     }
1312     /// <summary>优先当前高亮列，否则从左到右第一列排满的。</summary>
1313     public int ReadyColumn()
1314     {
1315         if (Won) return -1;
1316         if (IsFullUniform(Selected)) return Selected;
1317         for (int c = 0; c < ColumnCount; c++)
1318         {
1319             if (IsFullUniform(c)) return c;
1320         }
1321         return -1;
1322     }
1323     public bool CanPack => ReadyColumn() >= 0;
1324     public PackResult Pack() => PackAt(ReadyColumn());
1325     PackResult PackAt(int column)
1326     {
1327         var result = new PackResult { Column = -1, Type = -1 };
1328         if (column < 0 || !IsFullUniform(column)) return result;
1329         List<Cell> list = columns[column];
1330         result.Column = column;
1331         result.Type = list[0].Type;
1332         list.Clear();
1333         BoxFilled++;
1334         if (BoxFilled >= BoxSlots)
1335         {
1336             BoxFilled = 0;
1337             BoxesDone++;
1338             result.Shipped = true;
1339             if (BoxesDone >= 1)
1340                 open[ProgressColumn] = true;
1341             if (BoxesDone >= BoxesToWin)
1342             {
1343                 Won = true;
1344                 result.Won = true;
1345             }
1346         }
1347         return result;
1348     }
1349     public bool HasOpenSpace
1350     {
```

```
1351     get
1352     {
1353         for (int c = 0; c < ColumnCount; c++)
1354         {
1355             if (IsOpen(c) && columns[c].Count < ColumnHeight) return true;
1356         }
1357         return false;
1358     }
1359 }
1360 /// <summary>队列里还有这一局的裙子，并且已开的列上有空位，就可以补。不用等台上清空。</summary>
1361 public bool CanRefill => !Won && reserve.Count > 0 && HasOpenSpace;
1362 /// <summary>
1363 /// 每个还有空位的已开列叠一件。满列跳过，原来的衣服留在下面。
1364 /// 补上来的都是开局定好的那几种。
1365 /// </summary>
1366 public int Refill()
1367 {
1368     if (!CanRefill) return 0;
1369     int placed = 0;
1370     for (int c = 0; c < ColumnCount; c++)
1371     {
1372         if (!IsOpen(c) || columns[c].Count >= ColumnHeight) continue;
1373         if (reserve.Count == 0) break;
1374         PlaceIncoming(c, Take());
1375         placed++;
1376     }
1377     RevealReady();
1378     return placed;
1379 }
1380 public bool IsStuck
1381 {
1382     get
1383     {
1384         if (Won || CanPack || CanRefill) return false;
1385         for (int from = 0; from < ColumnCount; from++)
1386         {
1387             for (int to = 0; to < ColumnCount; to++)
1388             {
1389                 if (CanMove(from, to)) return false;
1390             }
1391         }
1392         return true;
1393     }
1394 }
1395 public static string SelfCheck()
1396 {
1397     for (int palette = 1; palette <= 4; palette++)
1398     {
1399         for (int seed = 1; seed <= 12; seed++)
1400         {
```

```

1401     var dealt = new PackBoard(palette, seed * 17 + palette);
1402     if (!dealt.Proven) return "unproven " + palette + ":" + seed;
1403     if (dealt.Column(2).Count != 0) return "parking " + palette;
1404     int onBoard = 0;
1405     for (int c = 0; c < ColumnCount; c++)
1406         onBoard += dealt.Column(c).Count;
1407     if (onBoard != ColumnHeight * 2) return "board " + onBoard;
1408     if (dealt.ReserveCount != ColumnHeight * 2 * 5) return "reserve " + dealt.Res...
1409     if (!dealt.CanRefill) return "no refill";
1410     if (palette > 1 && dealt.CanPack) return "already sorted";
1411     var seen = new bool[palette];
1412     for (int c = 0; c < 2; c++)
1413     {
1414         for (int i = 0; i < dealt.Column(c).Count; i++)
1415             seen[dealt.Column(c)[i].Type] = true;
1416     }
1417     if (dealt.Refill() != 1 || dealt.Column(2).Count != 1) return "gap";
1418     int added = dealt.Column(2)[0].Type;
1419     if (added < 0 || added >= palette || !seen[added]) return "new dress";
1420 }
1421 }
1422 var again = new PackBoard(4, 42);
1423 var same = new PackBoard(4, 42);
1424 for (int c = 0; c < StartOpen; c++)
1425 {
1426     if (again.Column(c).Count != same.Column(c).Count) return "seed len";
1427     for (int i = 0; i < again.Column(c).Count; i++)
1428     {
1429         if (again.Column(c)[i].Type != same.Column(c)[i].Type) return "seed";
1430     }
1431 }
1432 var board = new PackBoard(1, 3);
1433 if (!board.Proven) return "deal";
1434 if (board.Selected != -1) return "preselected";
1435 if (!board.IsOpen(0) || board.IsOpen(ProgressColumn) || board.IsOpen(AdColumnA))
1436     return "open";
1437 if (board.Column(0).Count != ColumnHeight) return "deal " + board.Column(0).Count;
1438 if (!board.Column(0)[0].Revealed) return "top hidden";
1439 if (board.Tap(0) != TapResult.Selected || board.Selected != 0) return "select";
1440 if (board.Tap(0) != TapResult.Selected || board.Selected != -1) return "deselect";
1441 // 最上面两件同款倒到另一列的顶上，底下不同款留着。空列也能接。
1442 board.SetColumn(0, 1, 1, 0);
1443 board.SetColumn(1, 1, 2);
1444 board.SetColumn(2);
1445 if (board.Tap(0) != TapResult.Selected) return "arm";
1446 if (board.Tap(1) != TapResult.Moved) return "pour";
1447 if (board.Column(0).Count != 1 || board.Column(0)[0].Type != 0) return "src left";
1448 if (board.Column(1).Count != 4
1449     || board.Column(1)[0].Type != 1
1450     || board.Column(1)[2].Type != 1

```

```

1451         || board.Column(1)[3].Type != 2)
1452         return "dst order";
1453     if (board.Selected != -1) return "still selected";
1454     board.SetColumn(0, 0, 0);
1455     if (board.Tap(0) != TapResult.Selected) return "arm empty";
1456     if (board.Tap(2) != TapResult.Moved) return "empty dest";
1457     if (board.Column(2).Count != 2 || board.Column(2)[0].Type != 0) return "emptied";
1458     // 空位比这组少，就先把空位倒满。
1459     board.SetColumn(0, 0, 0, 0);
1460     board.SetColumn(1, 0, 0, 0, 0, 0);
1461     if (board.Tap(0) != TapResult.Selected) return "arm tight";
1462     if (board.PourCount(0, 1) != 2) return "partial " + board.PourCount(0, 1);
1463     if (board.Tap(1) != TapResult.Moved || board.Column(0).Count != 1) return "split";
1464     if (board.Column(1).Count != ColumnHeight) return "filled";
1465     // 顶上不同款时不倒，第二次点击改成选中这一列。
1466     board.SetColumn(0, 1);
1467     board.SetColumn(1, 0);
1468     if (board.Tap(0) != TapResult.Selected) return "arm mismatch";
1469     if (board.Tap(1) != TapResult.Selected || board.Column(0).Count != 1) return "mismatch";
1470     board.SetColumn(0, 0, 0, 0, 0, 0, 0, 0);
1471     board.SetColumn(1, 0);
1472     if (board.Tap(1) != TapResult.Selected) return "arm full";
1473     if (board.ReadyColumn() != 0) return "ready " + board.ReadyColumn();
1474     PackResult packed = board.Pack();
1475     if (packed.Column != 0 || packed.Type != 0 || board.Column(0).Count != 0)
1476         return "pack";
1477     // 台上还有衣服也能补。每个空位叠一件，底下原来的留着。
1478     board.SetColumn(0, 1, 1);
1479     board.SetColumn(1);
1480     board.SetColumn(2);
1481     if (!board.CanRefill) return "refill";
1482     if (board.Refill() != 3) return "placed";
1483     if (board.Column(0).Count != 3 || board.Column(0)[2].Type != 1) return "kept";
1484     if (!board.UnlockAd(AdColumnA) || !board.IsOpen(AdColumnA)) return "ad";
1485     if (board.UnlockAd(ProgressColumn)) return "progress unlocked early";
1486     EnsureCatalog();
1487     var scripted = new PackBoard();
1488     var fallback = scripted.MakeScreen(0, 1, new Random(1), true);
1489     var rest = new List<int> { 0, 0, 1, 1, 0, 1, 0, 1, 0, 1 };
1490     if (!scripted.Play(fallback, rest)) return "fallback";
1491     return "ok";
1492 }
1493 void SetColumn(int column, params int[] types)
1494 {
1495     columns[column].Clear();
1496     for (int i = 0; i < types.Length; i++)
1497         columns[column].Add(new Cell { Type = types[i], Revealed = true });
1498 }
1499 struct DealScreen
1500 {

```

```
10170     if (!ok)
10171     {
10172         FailedAt[name] = Time.unscaledTime;
10173         Debug.LogWarning("[一裙又一裙] 立绘没拉到 " + name + " " + req.error);
10174     }
10175     req.Dispose();
10176     running--;
10177     if (Waiting.TryGetValue(name, out var list))
10178     {
10179         Waiting.Remove(name);
10180         if (ok)
10181         {
10182             for (int i = 0; i < list.Count; i++)
10183                 list[i]?.Invoke();
10184         }
10185     }
10186     Pump();
10187 }
10188 static string LocalUrl(string name)
10189 {
10190     if (!Application.isEditor) return null;
10191     string path = Path.GetFullPath(Path.Combine(Application.dataPath, "DressSort/Art", na...
10192     if (!File.Exists(path)) return null;
10193     return new Uri(path).AbsoluteUri;
10194 }
10195 static readonly List<UnityEngine.Object> Dead = new List<UnityEngine.Object>();
10196 static void Trim()
10197 {
10198     for (int i = 0; i < Recent.Count && Sprites.Count > MaxTextures;)
10199     {
10200         string name = Recent[i];
10201         Sprite s = Sprites[name];
10202         if (InUse(s)) { i++; continue; }
10203         Recent.RemoveAt(i);
10204         Sprites.Remove(name);
10205         if (s != null)
10206         {
10207             if (s.texture != null) Destroy(s.texture);
10208             Destroy(s);
10209         }
10210     }
10211 }
10212 static bool InUse(Sprite s)
10213 {
10214     bool used = false;
10215     Dead.Clear();
10216     foreach (var kv in Bound)
10217     {
10218         if (kv.Key == null) { Dead.Add(kv.Key); continue; }
10219         if (kv.Key is Image img && img.sprite == s) used = true;
```

```
10220     }
10221     for (int i = 0; i < Dead.Count; i++) Bound.Remove(Dead[i]);
10222     return used;
10223 }
10224 }
10225 }
10226 // ===== FILE: Assets/DressSort/Scripts/Platform/CdnManifest.cs =====
10227 // 由 Tools/cdn_build.py 生成, 别手改。
10228 using System.Collections.Generic;
10229 namespace DressSort
10230 {
10231     public static class CdnManifest
10232     {
10233         public const string BaseUrl = "https://726f-rosa-env-d7grf78r5dbd37323-1414200063.tcb.qcl...
10234         // 逻辑名 -> 云上相对路径
10235         public static readonly Dictionary<string, string> Files =
10236             new Dictionary<string, string>
10237             {
10238                 { "Audio/bgm_event", "Audio/bgm_event_e2382b6b9cec45ac635d525b7a0ff8a1.mp3" },
10239                 { "Audio/bgm_lobby", "Audio/bgm_lobby_b0b8c122648b30d41ff1b1bf501fad46.mp3" },
10240                 { "Audio/bgm_play", "Audio/bgm_play_6844523a5638fef0840cb2af8ad28cce.mp3" },
10241                 { "Portraits/body_black_ribbon", "Portraits/body_black_ribbon_e0bfd5fe17bd6068a75...
10242                 { "Portraits/body_blueberry", "Portraits/body_blueberry_2a3d620c30ce78936f00fa1af...
10243                 { "Portraits/body_cherry_red", "Portraits/body_cherry_red_ea0fce4061f6ea7dd40d601...
10244                 { "Portraits/body_coral_sun", "Portraits/body_coral_sun_887bda1af69e6ca374dbf0da4...
10245                 { "Portraits/body_cream_cloud", "Portraits/body_cream_cloud_9872f567080285f56bda5...
10246                 { "Portraits/body_grape_school", "Portraits/body_grape_school_0e55fb0f4d0b9da90a8...
10247                 { "Portraits/body_honey_bow", "Portraits/body_honey_bow_8353d77483c5f737a36d47815...
10248                 { "Portraits/body_ivory_lace", "Portraits/body_ivory_lace_462210bb84f33babe9248ca...
10249                 { "Portraits/body_lavender_star", "Portraits/body_lavender_star_1959ddf06a2077104...
10250                 { "Portraits/body_lemon_print", "Portraits/body_lemon_print_bc5e909bfe7aa10f2e4ae...
10251                 { "Portraits/body_lilac_lace", "Portraits/body_lilac_lace_c51b716bb4d52a5446a2af9...
10252                 { "Portraits/body_matcha_pinafore", "Portraits/body_matcha_pinafore_57bb8058efb69...
10253                 { "Portraits/body_mint_bow", "Portraits/body_mint_bow_b63fc37f76f93b785368d9b7ea9...
10254                 { "Portraits/body_navy_stripe", "Portraits/body_navy_stripe_1911b092f2627610699df...
10255                 { "Portraits/body_orange_slice", "Portraits/body_orange_slice_f8d2d29d7a8cca775da...
10256                 { "Portraits/body_peach_puff", "Portraits/body_peach_puff_bf079a0ee2c8a3dd83f39f4...
10257                 { "Portraits/body_pink_gingham", "Portraits/body_pink_gingham_6e152a526bb73a68075...
10258                 { "Portraits/body_rose_velvet", "Portraits/body_rose_velvet_bba74ea8a3aa1db19caf...
10259                 { "Portraits/body_sky_dot", "Portraits/body_sky_dot_9e1e01afd82815b303a86c33f6a77...
10260                 { "Portraits/body_strawberry", "Portraits/body_strawberry_c7dd22ca9d6eb500b7eae3d...
10261                 { "Portraits/body_tea_sailor", "Portraits/body_tea_sailor_4a9f3c76051c89dc0229a...
10262                 { "Portraits/hairback_milktea_long", "Portraits/hairback_milktea_long_ae7c769bd8a...
10263                 { "Portraits/head_apricot", "Portraits/head_apricot_2dd4a6cd1fb4859348011beae4743...
10264                 { "Portraits/head_baguette", "Portraits/head_baguette_cdaf80f8c58ad907b2000d2bf53...
10265                 { "Portraits/head_caramel", "Portraits/head_caramel_163390e880193dcaaf704617362c5...
10266                 { "Portraits/head_denim", "Portraits/head_denim_d304002b0d9715236836298f64914f27...
10267                 { "Portraits/head_milktea", "Portraits/head_milktea_184451c0dfa7f715a306489557105...
10268                 { "Portraits/head_milktea_long", "Portraits/head_milktea_long_636d050f5e8b6314bad...
10269                 { "Portraits/head_pink", "Portraits/head_pink_a211a4829bc0ec247bbeff5c34bacfe4.pn...
```

```
10270         { "Portraits/head_wisteria", "Portraits/head_wisteria_f1d4872181697741d5ff7a1d755...
10271     };
10272     public static string Url(string name)
10273     {
10274         string rel;
10275         return Files.TryGetValue(name, out rel) ? BaseUrl + rel : null;
10276     }
10277 }
10278 }
10279 // ===== FILE: Assets/DressSort/Scripts/Platform/CdnPortraits.cs =====
10280 using System.Collections.Generic;
10281 namespace DressSort
10282 {
10283     /// <summary>
10284     /// 进游戏先拉开局装扮、棋盘 8 款和前 20 关直送。
10285     /// 更后面的衣服等换上那件时再拉，并且插到队首。
10286     /// </summary>
10287     public static class CdnPortraits
10288     {
10289         public static void Warm()
10290         {
10291             var earlyIds = new List<string>();
10292             earlyIds.Add("teal_sailor");
10293             earlyIds.Add("hair_milktea");
10294             LevelCatalog.CopyBoardDressIds(earlyIds);
10295             CraftCatalog.CopyGiftIds(earlyIds);
10296             var early = new List<string>();
10297             for (int i = 0; i < earlyIds.Count; i++)
10298                 AddKeys(early, earlyIds[i]);
10299             for (int i = 0; i < early.Count; i++)
10300                 CdnAssets.Prefetch(early[i]);
10301         }
10302         public static string KeyFor(UnityEngine.Sprite sprite)
10303         {
10304             if (sprite == null || string.IsNullOrEmpty(sprite.name)) return null;
10305             return "Portraits/" + sprite.name;
10306         }
10307         static void AddKeys(List<string> keys, string id)
10308         {
10309             if (string.IsNullOrEmpty(id)) return;
10310             if (id.StartsWith("hair_"))
10311             {
10312                 string key = id.Substring(5);
10313                 keys.Add("Portraits/head_" + key);
10314                 keys.Add("Portraits/hairback_" + key);
10315                 return;
10316             }
10317             if (id.StartsWith("wing_")) return;
10318             keys.Add("Portraits/body_" + id);
10319         }
10319     }
```

```
10320     }
10321 }
10322 // ===== FILE: Assets/DressSort/Scripts/Platform/CloudConfig.cs =====
10323 namespace DressSort
10324 {
10325     /// <summary>
10326     /// 后端是 CloudBase 云函数 cloudfunctions/dresssort-api, 和 black-rosa 同一个环境。
10327     /// GameKey、云函数目录名、云端 GAME_KEY 环境变量三处必须一致。
10328     /// 真机要把 BaseUrl 的域名加进小游戏后台的 request 合法域名。
10329     /// </summary>
10330     public static class CloudConfig
10331     {
10332         public const string GameKey = "dresssort";
10333         public const string BaseUrl = "https://rosa-env-d7grf78r5dbd37323.service.tcloudbase.com";
10334         const string Prefix = "/" + GameKey + "-api";
10335         public const string LoginPath = Prefix + "/login";
10336         public const string RankSubmitPath = Prefix + "/rank/submit";
10337         public const string RankListPath = Prefix + "/rank/list";
10338         public const string GameClubDailyPath = Prefix + "/gameclub/daily";
10339         public const int RankListLimit = 50;
10340         public const int RequestTimeoutSec = 10;
10341         public const string TokenKey = GameKey + "_token";
10342         public const string AnonKey = GameKey + "_anon_id";
10343         public const string ProfileKey = GameKey + "_wx_profile";
10344         public const string RankSentKey = GameKey + "_rank_sent";
10345     }
10346 }
10347 // ===== FILE: Assets/DressSort/Scripts/Platform/GameClubService.cs =====
10348 using System;
10349 using System.Collections;
10350 using UnityEngine;
10351 namespace DressSort
10352 {
10353     /// <summary>
10354     /// 游戏圈每日发帖: wx.getGameClubData 拿加密的「今日发帖数」,
10355     /// 云函数用登录时存下的 session_key 解开。session_key 过期或还没存就重登一次再拉。
10356     /// 编辑器里没有游戏圈, 当作已发帖, 方便验领奖。
10357     /// </summary>
10358     public sealed class GameClubService : MonoBehaviour
10359     {
10360         const int DailyPostType = 6;
10361         [Serializable]
10362         sealed class DailyBody
10363         {
10364             public string encryptedData;
10365             public string iv;
10366         }
10367         static GameClubService _it;
10368         static bool _hooked;
10369         public static event Action Shown;
```

```

10370     static GameClubService It
10371     {
10372         get
10373         {
10374             if (_it != null) return _it;
10375             var go = new GameObject("GameClubService");
10376             if (Application.isPlaying) DontDestroyOnLoad(go);
10377             _it = go.AddComponent<GameClubService>();
10378             return _it;
10379         }
10380     }
10381     /// <summary>从游戏圈切回游戏时通知。只注册一次，面板自己订阅 / 退订 Shown。</summary>
10382     public static void Hook()
10383     {
10384         if (_hooked) return;
10385         _hooked = true;
10386         WxBridge.OnShow() => Shown?.Invoke();
10387     }
10388     /// <summary>done(今日发帖数, err): err 为 null 表示查到了。</summary>
10389     public static void DailyPosts(Action<int, string> done)
10390     {
10391         if (!WxBridge.CanUseClub)
10392         {
10393             if (Application.isEditor) done(1, null);
10394             else done(0, "not wx");
10395             return;
10396         }
10397         It.StartCoroutine(It.Fetch(done));
10398     }
10399     IEnumerator Fetch(Action<int, string> done)
10400     {
10401         if (!WxBridge.PrivacyAgreed)
10402         {
10403             bool settled = false, agreed = false;
10404             WxBridge.EnsurePrivacy(ok => { agreed = ok; settled = true; });
10405             float privacyUntil = Time.realtimeSinceStartup + 20f;
10406             while (!settled && Time.realtimeSinceStartup < privacyUntil) yield return null;
10407             if (!agreed)
10408             {
10409                 done(0, "需同意隐私协议后才能同步游戏圈发帖");
10410                 yield break;
10411             }
10412         }
10413         for (int attempt = 0; attempt < 2; attempt++)
10414         {
10415             string enc = null, iv = null, err = null;
10416             bool back = false;
10417             WxBridge.GameClubData(DailyPostType, (e, i, x) => { enc = e; iv = i; err = x; bac...
10418             float until = Time.realtimeSinceStartup + CloudConfig.RequestTimeoutSec;
10419             while (!back && Time.realtimeSinceStartup < until) yield return null;

```

```

10420         if (!back) { done(0, "getGameClubData timeout"); yield break; }
10421         if (err != null) { done(0, err); yield break; }
10422         CloudReply reply = null;
10423         string callErr = null;
10424         string json = JsonUtility.ToJson(new DailyBody { encryptedData = enc, iv = iv });
10425         yield return Backend.Call(CloudConfig.GameClubDailyPath, json, (s, r, e) => { rep...
10426         if (callErr == null)
10427         {
10428             done(reply.data != null ? reply.data.postCount : 0, null);
10429             yield break;
10430         }
10431         string code = reply != null ? reply.code : null;
10432         bool stale = code == "NO_WX_SESSION" || code == "DECRYPT_FAIL";
10433         if (!stale || attempt > 0) { done(0, callErr); yield break; }
10434         // 先重登换新 session_key, 再重新拉加密数据; 顺序反了新数据还是旧 key 加的密。
10435         Backend.ClearToken();
10436         string loginErr = null;
10437         yield return Backend.EnsureToken(e => loginErr = e);
10438         if (loginErr != null) { done(0, loginErr); yield break; }
10439     }
10440 }
10441 }
10442 }
10443 // ===== FILE: Assets/DressSort/Scripts/Platform/RankService.cs =====
10444 using System;
10445 using System.Collections;
10446 using System.Collections.Generic;
10447 using UnityEngine;
10448 using UnityEngine.Networking;
10449 namespace DressSort
10450 {
10451     /// <summary>
10452     /// 通关排行榜: 每次通通报一次已通关关数, 同样的「关数 + 昵称头像」只报一次;
10453     /// 打开榜单先把没报上去的补报, 再拉前 N 名和自己的名次。
10454     /// 服务端只升不降, 本地清档不会把榜上成绩拉下来。
10455     /// </summary>
10456     public sealed class RankService : MonoBehaviour
10457     {
10458         [Serializable]
10459         sealed class SubmitBody
10460         {
10461             public int cleared;
10462             public string displayName;
10463             public string avatarUrl;
10464         }
10465         [Serializable]
10466         sealed class ListBody
10467         {
10468             public int limit;
10469         }

```

```

10470 [Serializable]
10471 sealed class Profile
10472 {
10473     public string nick;
10474     public string avatar;
10475 }
10476 public sealed class Board
10477 {
10478     public RankRow[] List;
10479     public RankRow Mine;
10480 }
10481 static RankService _it;
10482 static Profile _profile;
10483 readonly Dictionary<string, Sprite> _avatars = new Dictionary<string, Sprite>();
10484 readonly Dictionary<string, Action<Sprite>> _avatarWait = new Dictionary<string, Action<S...
10485 bool _sending;
10486 static RankService It
10487 {
10488     get
10489     {
10490         if (_it != null) return _it;
10491         var go = new GameObject("RankService");
10492         if (Application.isPlaying) DontDestroyOnLoad(go);
10493         _it = go.AddComponent<RankService>();
10494         return _it;
10495     }
10496 }
10497 public static bool HasProfile => !string.IsNullOrEmpty(LoadProfile().nick);
10498 public static string Nick => LoadProfile().nick ?? "";
10499 public static string Avatar => LoadProfile().avatar ?? "";
10500 static Profile LoadProfile()
10501 {
10502     if (_profile != null) return _profile;
10503     string raw = PlayerPrefs.GetString(CloudConfig.ProfileKey, "");
10504     try { _profile = string.IsNullOrEmpty(raw) ? null : JsonUtility.FromJson<Profile>(raw...
10505     catch (Exception) { _profile = null; }
10506     if (_profile == null) _profile = new Profile();
10507     return _profile;
10508 }
10509 <summary>微信拒绝授权也会回一个「微信用户」+ 灰头像，当没给。</summary>
10510 public static bool SetProfile(string nick, string avatar)
10511 {
10512     nick = (nick ?? "").Trim();
10513     if (nick.Length == 0 || nick == "微信用户") return false;
10514     _profile = new Profile { nick = nick, avatar = avatar ?? "" };
10515     PlayerPrefs.SetString(CloudConfig.ProfileKey, JsonUtility.ToJson(_profile));
10516     PlayerPrefs.Save();
10517     return true;
10518 }
10519 static string SentKey(int cleared) => cleared + ":" + Nick + ":" + Avatar;

```

```

10520     public static void Submit(int cleared)
10521     {
10522         if (cleared <= 0 || !Application.isPlaying) return;
10523         if (PlayerPrefs.GetString(CloudConfig.RankSentKey, "") == SentKey(cleared)) return;
10524         It.StartCoroutine(It.SubmitRoutine(cleared));
10525     }
10526     public static void Load(int cleared, Action<Board, string> done)
10527     {
10528         It.StartCoroutine(It.LoadRoutine(cleared, done));
10529     }
10530     IEnumerator SubmitRoutine(int cleared)
10531     {
10532         while (_sending) yield return null;
10533         string key = SentKey(cleared);
10534         if (cleared <= 0 || PlayerPrefs.GetString(CloudConfig.RankSentKey, "") == key) yield ...
10535         _sending = true;
10536         var body = new SubmitBody { cleared = cleared, displayName = Nick, avatarUrl = Avatar };
10537         string err = null;
10538         yield return Backend.Call(CloudConfig.RankSubmitPath, JsonUtility.ToJson(body), (s, r...
10539         _sending = false;
10540         if (err == null)
10541         {
10542             PlayerPrefs.SetString(CloudConfig.RankSentKey, key);
10543             PlayerPrefs.Save();
10544         }
10545         else Debug.LogWarning("[Rank] submit failed: " + err);
10546     }
10547     IEnumerator LoadRoutine(int cleared, Action<Board, string> done)
10548     {
10549         yield return SubmitRoutine(cleared);
10550         CloudReply reply = null;
10551         string err = null;
10552         var body = new ListBody { limit = CloudConfig.RankListLimit };
10553         yield return Backend.Call(CloudConfig.RankListPath, JsonUtility.ToJson(body), (s, r, ...
10554         if (err != null || reply == null || reply.data == null)
10555         {
10556             done(null, err ?? "empty reply");
10557             yield break;
10558         }
10559         // JsonUtility 不会把缺省的对象字段留成 null, 没上榜时 mine 是个 rank 为 0 的空壳。
10560         RankRow mine = reply.data.mine != null && reply.data.mine.rank > 0 ? reply.data.mine ...
10561         done(new Board { List = reply.data.list ?? new RankRow[0], Mine = mine }, null);
10562     }
10563     /// <summary>头像按 URL 缓存。微信头像域名要加进小游戏后台的 downloadFile 合法域名。</summary>
10564     public static void AvatarOf(string url, Action<Sprite> done)
10565     {
10566         if (string.IsNullOrEmpty(url)) { done(null); return; }
10567         var s = It;
10568         if (s._avatars.TryGetValue(url, out Sprite hit)) { done(hit); return; }
10569         if (s._avatarWait.ContainsKey(url)) { s._avatarWait[url] += done; return; }

```

```

10570         s._avatarWait[url] = done;
10571         s.StartCoroutine(s.FetchAvatar(url));
10572     }
10573     IEnumerator FetchAvatar(string url)
10574     {
10575         Sprite sprite = null;
10576         using (var req = UnityWebRequestTexture.GetTexture(url, true))
10577         {
10578             req.timeout = CloudConfig.RequestTimeoutSec;
10579             yield return req.SendWebRequest();
10580             if (req.result == UnityWebRequest.Result.Success)
10581             {
10582                 Texture2D tex = DownloadHandlerTexture.GetContent(req);
10583                 if (tex != null)
10584                 {
10585                     tex.wrapMode = TextureWrapMode.Clamp;
10586                     sprite = Sprite.Create(tex, new Rect(0, 0, tex.width, tex.height), new Ve...
10587                 }
10588             }
10589         }
10590         if (sprite != null) _avatars[url] = sprite;
10591         if (_avatarWait.TryGetValue(url, out Action<Sprite> wait))
10592         {
10593             _avatarWait.Remove(url);
10594             wait?.Invoke(sprite);
10595         }
10596     }
10597 }
10598 }
10599 // ===== FILE: Assets/DressSort/Scripts/Platform/WxBridge.cs =====
10600 using System;
10601 using UnityEngine;
10602 namespace DressSort
10603 {
10604     /// <summary>
10605     /// 微信 SDK 用反射调用，编辑器里没有 Wx 程序集也能编译。
10606     /// 做法对齐 black-rosa：胶囊、安全区、触摸覆盖、启动保活。
10607     /// </summary>
10608     public static class WxBridge
10609     {
10610         static Type WxType()
10611         {
10612             return Type.GetType("WeChatWASM.WX, Wx") ?? Type.GetType("WeChatWASM.WX");
10613         }
10614         public static bool IsMiniGame
10615         {
10616             get
10617             {
10618                 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10619                     return true;

```

```

10620 #else
10621     string p = Application.platform.ToString();
10622     return p.IndexOf("MiniGame", StringComparison.OrdinalIgnoreCase) >= 0
10623         || p.IndexOf("Weixin", StringComparison.OrdinalIgnoreCase) >= 0;
10624 #endif
10625 }
10626 }
10627 public static void InitSdk(Action ready)
10628 {
10629     var wx = WxType();
10630     var init = wx != null ? wx.GetMethod("InitSDK", new[] { typeof(Action<int>) }): null;
10631     if (init == null)
10632     {
10633         ready?.Invoke();
10634         return;
10635     }
10636     Action<int> cb = _ => ready?.Invoke();
10637     init.Invoke(null, new object[] { cb });
10638 }
10639 public static void KeepRuntime()
10640 {
10641     WxType()?.GetMethod("GetSystemInfoSync", Type.EmptyTypes)?.Invoke(null, null);
10642 }
10643 public static void OverrideTouch(GameObject eventSystem)
10644 {
10645     var t = Type.GetType("WXTouchInputOverride, Wx") ?? Type.GetType("WXTouchInputOverrid...
10646     if (t == null || eventSystem == null || eventSystem.GetComponent(t) != null) return;
10647     eventSystem.AddComponent(t);
10648 }
10649 public static void ShowShareMenu()
10650 {
10651     var wx = WxType();
10652     wx?.GetMethod("ShowShareMenu", Type.EmptyTypes)?.Invoke(null, null);
10653 }
10654 /// <summary>wx.login 拿一次性 code, 换 openid 在云函数里做。非微信构建直接报失败。</summary>
10655 public static void Login(Action<string> ok, Action<string> fail)
10656 {
10657 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10658     WeChatWASM.WX.Login(new WeChatWASM.LoginOption
10659     {
10660         success = r => ok(r.code),
10661         fail = e => fail(e.errMsg)
10662     });
10663 #else
10664     fail("not minigame");
10665 #endif
10666 }
10667 /// <summary>从游戏圈切回来时刷新进度。编辑器里没有, 返回 false。</summary>
10668 public static bool OnShow(Action show)
10669 {

```

```

10670 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10671     if (Application.isEditor) return false;
10672     WeChatWASM.WX.OnShow(_ => show());
10673     return true;
10674 #else
10675     return false;
10676 #endif
10677 }
10678 /// <summary>真机上才有微信原生的授权按钮和游戏圈按钮。</summary>
10679 public static bool CanAskProfile
10680 {
10681     get
10682     {
10683 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10684         return !Application.isEditor;
10685 #else
10686         return false;
10687 #endif
10688     }
10689 }
10690 public static bool CanUseClub => CanAskProfile;
10691 /// <summary>
10692 /// 昵称和游戏圈数据都要先过隐私授权，才会弹出微信自己的隐私弹窗。
10693 /// 同意过的这次会话里不再问。编辑器没有这套接口，直接当已同意。
10694 /// </summary>
10695 public static bool PrivacyAgreed { get; private set; }
10696 static bool _privacyBusy;
10697 static Action<bool> _privacyWait;
10698 public static void EnsurePrivacy(Action<bool> done)
10699 {
10700     if (!CanAskProfile || PrivacyAgreed)
10701     {
10702         if (!CanAskProfile) PrivacyAgreed = true;
10703         done?.Invoke(true);
10704         return;
10705     }
10706     _privacyWait += done;
10707     if (_privacyBusy) return;
10708     _privacyBusy = true;
10709 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10710     try
10711     {
10712         WeChatWASM.WX.RequirePrivacyAuthorize(new WeChatWASM.RequirePrivacyAuthorizeOption
10713         {
10714             success = _ => FinishPrivacy(true),
10715             fail = _ => FinishPrivacy(false),
10716         });
10717     }
10718     catch (Exception e)
10719     {

```

```

10720         Debug.LogWarning("[Privacy] " + e.Message);
10721         FinishPrivacy(true);
10722     }
10723 #else
10724     FinishPrivacy(true);
10725 #endif
10726 }
10727 static void FinishPrivacy(bool ok)
10728 {
10729     _privacyBusy = false;
10730     if (ok) PrivacyAgreed = true;
10731     Action<bool> wait = _privacyWait;
10732     _privacyWait = null;
10733     wait?.Invoke(ok);
10734 }
10735 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10736     static WeChatWASM.WXUserInfoButton _infoBtn;
10737     static WeChatWASM.WXGameClubButton _clubBtn;
10738 #endif
10739     /// <summary>
10740     /// 微信只允许用它自己画的透明按钮拿昵称头像，所以在 Unity 按钮正上方盖一个同样大小的。
10741     /// SDK 里会把坐标再除一次 devicePixelRatio，所以这里传物理像素。
10742     /// got(nick, avatarUrl, err): 没拿到时 nick 为空。
10743     /// </summary>
10744     public static void ShowProfileButton(Rect screenRect, Action<string, string, string> got)
10745     {
10746         HideProfileButton();
10747 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10748         if (Application.isEditor) return;
10749         RectInt r = ToPhysical(screenRect);
10750         _infoBtn = WeChatWASM.WX.CreateUserInfoButton(r.x, r.y, r.width, r.height, "zh_CN", f...
10751         _infoBtn.OnTap(res =>
10752         {
10753             // userInfo 是结构体，不能和 null 写在同一个三元表达式里。
10754             if (res == null || string.IsNullOrEmpty(res.userInfo.nickName))
10755             {
10756                 string err = res == null ? "no response" : (res.errCode + " " + res.errMsg);
10757                 got(null, null, err);
10758                 return;
10759             }
10760             got(res.userInfo.nickName, res.userInfo.avatarUrl ?? "", null);
10761         });
10762         _infoBtn.Show();
10763 #endif
10764     }
10765     public static void HideProfileButton()
10766     {
10767 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10768         if (_infoBtn == null) return;
10769         _infoBtn.Destroy();

```

```

10770     _infoBtn = null;
10771 #endif
10772 }
10773 ///

```

```

10820         left = r.x,
10821         top = r.y,
10822         width = r.width,
10823         height = r.height,
10824         backgroundColor = "rgba(0,0,0,0)",
10825         borderColor = "rgba(0,0,0,0)",
10826         borderWidth = 0,
10827         borderRadius = 0,
10828         color = "rgba(0,0,0,0)",
10829         fontSize = 12,
10830         lineHeight = r.height,
10831     },
10832 });
10833 _clubBtn?.Show();
10834 #endif
10835 }
10836 public static void HideClubButton()
10837 {
10838 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10839     if (_clubBtn == null) return;
10840     _clubBtn.Destroy();
10841     _clubBtn = null;
10842 #endif
10843 }
10844 /// <summary>游戏圈数据是加密的，用登录时的 session_key 在云函数里解。done(encryptedData, iv, err)。</summary>
10845 public static void GameClubData(int dataType, Action<string, string, string> done)
10846 {
10847 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10848     if (!Application.isEditor)
10849     {
10850         WeChatWASM.WX.GetGameClubData(new WeChatWASM.GetGameClubDataOption
10851         {
10852             dataTypeList = new[] { new WeChatWASM.DataType { type = dataType } },
10853             success = r => done(r.encryptedData, r.iv, string.IsNullOrEmpty(r.encryptedDa...
10854             fail = e => done(null, null, e != null ? e.errMsg : "fail"),
10855         });
10856         return;
10857     }
10858 #endif
10859     done(null, null, "not minigame");
10860 }
10861 static Font _sysFont;
10862 static bool _sysFontAsked;
10863 static Action<Font> _sysFontWait;
10864 /// <summary>
10865 /// 玩家昵称什么字都可能，游戏字体只切了用到的几百个字。
10866 /// 真机拿微信的系统字体；编辑器里用本机字体。拿不到回调 null。
10867 /// </summary>
10868 public static void SystemFont(Action<Font> done)
10869 {

```

```

10870         if (_sysFont != null || (_sysFontAsked && _sysFontWait == null)) { done(_sysFont); re...
10871         _sysFontWait += done;
10872         if (_sysFontAsked) return;
10873         _sysFontAsked = true;
10874 #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10875         if (!Application.isEditor)
10876         {
10877             WeChatWASM.WX.GetWXFont(null, f => FinishFont(f));
10878             return;
10879         }
10880 #endif
10881         FinishFont(Font.CreateDynamicFontFromOSFont(
10882             new[] { "PingFang SC", "Heiti SC", "Noto Sans CJK SC", "Arial Unicode MS" }, 28));
10883     }
10884     static void FinishFont(Font f)
10885     {
10886         _sysFont = f;
10887         Action<Font> wait = _sysFontWait;
10888         _sysFontWait = null;
10889         wait?.Invoke(f);
10890     }
10891     /// <summary>游戏圈按钮的 style 是逻辑像素, SDK 不再缩放。</summary>
10892     static RectInt ToWindow(Rect screenRect)
10893     {
10894         float winW = Num(SystemInfo(), "windowWidth");
10895         float k = winW > 0f ? winW / Mathf.Max(1, Screen.width) : 1f;
10896         return TopLeft(screenRect, k);
10897     }
10898     /// <summary>昵称按钮的 JS 会再除 devicePixelRatio。Unity 屏幕若已是逻辑像素, 这里先乘回去。</summary>
10899     static RectInt ToPhysical(Rect screenRect)
10900     {
10901         float winW = Num(SystemInfo(), "windowWidth");
10902         float dpr = Num(SystemInfo(), "pixelRatio");
10903         if (dpr < 1f) dpr = 1f;
10904         float scale = winW > 0f && Screen.width < winW * 1.5f ? dpr : 1f;
10905         return TopLeft(screenRect, scale);
10906     }
10907     static RectInt TopLeft(Rect screenRect, float scale)
10908     {
10909         return new RectInt(
10910             Mathf.RoundToInt(screenRect.xMin * scale),
10911             Mathf.RoundToInt((Screen.height - screenRect.yMax) * scale),
10912             Mathf.Max(1, Mathf.RoundToInt(screenRect.width * scale)),
10913             Mathf.Max(1, Mathf.RoundToInt(screenRect.height * scale)));
10914     }
10915     /// <summary>UI 节点在屏幕上的像素矩形, 左下为原点。原生按钮按它摆位置。</summary>
10916     public static Rect ScreenRect(RectTransform rect)
10917     {
10918         var corners = new Vector3[4];
10919         rect.GetWorldCorners(corners);

```

```

10920     Canvas canvas = rect.GetComponentInParent<Canvas>();
10921     Camera cam = canvas != null && canvas.renderMode != RenderMode.ScreenSpaceOverlay
10922         ? canvas.worldCamera : null;
10923     Vector2 a = RectTransformUtility.WorldToScreenPoint(cam, corners[0]);
10924     Vector2 b = RectTransformUtility.WorldToScreenPoint(cam, corners[2]);
10925     return Rect.MinMaxRect(a.x, a.y, b.x, b.y);
10926 }
10927 static float _capsule = float.NaN;
10928 public static float CapsuleBottomFrac()
10929 {
10930     if (!float.IsNaN(_capsule)) return _capsule;
10931     _capsule = IsMiniGame ? 0.095f : -1f;
10932     try
10933     {
10934         object rect = WxType()?.GetMethod("GetMenuButtonBoundingClientRect", Type.EmptyTy...
10935             ?.Invoke(null, null);
10936         float bottom = Num(rect, "bottom");
10937         float screenH = Num(SystemInfo(), "screenHeight");
10938         if (bottom > 0f && screenH > 0f) _capsule = bottom / screenH;
10939     }
10940     catch (Exception)
10941     {
10942     }
10943     return _capsule;
10944 }
10945 static Vector2 _inset = new Vector2(float.NaN, float.NaN);
10946 public static Vector2 SafeInsetFrac()
10947 {
10948     if (!float.IsNaN(_inset.x)) return _inset;
10949     _inset = new Vector2(-1f, -1f);
10950     try
10951     {
10952         object sys = SystemInfo();
10953         float screenH = Num(sys, "screenHeight");
10954         object area = Member(sys, "safeArea");
10955         float top = Num(area, "top");
10956         float bottom = Num(area, "bottom");
10957         if (screenH > 0f && bottom > top && bottom <= screenH)
10958             _inset = new Vector2(top / screenH, (screenH - bottom) / screenH);
10959     }
10960     catch (Exception)
10961     {
10962     }
10963     return _inset;
10964 }
10965 static object SystemInfo()
10966 {
10967     return WxType()?.GetMethod("GetSystemInfoSync", Type.EmptyTypes)?.Invoke(null, null);
10968 }
10969 /// <summary>

```

```

10970    /// 激励视频。广告位还没配、或在编辑器里，直接当看完。
10971    /// 配了广告位才走微信 CreateRewardedVideoAd，看完才算成功。
10972    /// </summary>
10973    public const string RewardedAdUnitId = "";
10974    public static void ShowRewarded(Action success, Action fail)
10975    {
10976        if (string.IsNullOrEmpty(RewardedAdUnitId) || Application.isEditor)
10977        {
10978            success?.Invoke();
10979            return;
10980        }
10981    #if UNITY_MINIGAME || WEIXINMINIGAME || UNITY_WEIXINMINIGAME || MINIGAME_SUBPLATFORM_WEIXIN
10982        try
10983        {
10984            WeChatWASM.WXRewardedVideoAd ad = WeChatWASM.WX.CreateRewardedVideoAd(
10985                new WeChatWASM.WXCreateRewardedVideoAdParam { adUnitId = RewardedAdUnitId });
10986            bool settled = false;
10987            ad.OnClose(res =>
10988            {
10989                if (settled) return;
10990                settled = true;
10991                if (res != null && res.isEnded)
10992                    success?.Invoke();
10993                else
10994                    fail?.Invoke();
10995            });
10996            ad.Show(null, _ =>
10997            {
10998                if (settled) return;
10999                settled = true;
11000                fail?.Invoke();
11001            });
11002        }
11003        catch (Exception)
11004        {
11005            fail?.Invoke();
11006        }
11007    #else
11008        success?.Invoke();
11009    #endif
11010    }
11011    static object Member(object o, string name)
11012    {
11013        if (o == null) return null;
11014        Type t = o.GetType();
11015        var f = t.GetField(name);
11016        if (f != null) return f.GetValue(o);
11017        var p = t.GetProperty(name);
11018        return p?.GetValue(o);
11019    }

```

```

11020     static float Num(object o, string name)
11021     {
11022         object v = Member(o, name);
11023         return v == null ? 0f : Convert.ToSingle(v);
11024     }
11025 }
11026 public static class WxRuntimeKeep
11027 {
11028     [RuntimeInitializeOnLoadMethod(RuntimeInitializeLoadType.AfterAssembliesLoaded)]
11029     static void Keep()
11030     {
11031         Application.targetFrameRate = 60;
11032         Input.simulateMouseWithTouches = true;
11033         WxBridge.KeepRuntime();
11034     }
11035 }
11036 }
11037 // ===== FILE: cloudfunctions/dresssort-api/index.js =====
11038 /**
11039  * 一裙又一裙后端（CloudBase 云函数 + HTTP 访问服务），照 blackrosa-api。
11040  *
11041  * 路由（POST JSON）：
11042  * /login      wx code2session / anon, 签发 JWT
11043  * /rank/submit 上报已通关关数（只升不降），顺带更新昵称头像
11044  * /rank/list   通关榜前 N 名 + 自己的名次
11045  * /gameclub/daily 解密 wx.getGameClubData, 回当天游戏圈发帖数
11046  * /health      健康检查
11047  *
11048  * 环境变量:
11049  * GAME_KEY=dresssort
11050  * DRESSSORT_JWT_SECRET          必填
11051  * DRESSSORT_WX_APPID / DRESSSORT_WX_SECRET 微信 code2session
11052  * DRESSSORT_TOKEN_TTL_SEC        可选
11053  * DRESSSORT_RANK_MAX_CLEARED (默认 500) / DRESSSORT_RANK_BLOCKED_UIDS (逗号分隔) 可选
11054  *
11055  * 集合: dresssort_rankings (userId 唯一索引; cleared desc + reachedAt asc)
11056  *       dresssort_wxSessions (userId 唯一索引)
11057  */
11058 const { handleLogin } = require('./lib/auth');
11059 const { handleSubmit, handleList } = require('./lib/rank');
11060 const { handleDailyPost } = require('./lib/game-club');
11061 const { respond, parseEvent, preflight } = require('./lib/http');
11062 const ROUTES = {
11063     'GET /health': async () => ({ ok: true, ts: Date.now() }),
11064     'POST /health': async () => ({ ok: true, ts: Date.now() }),
11065     'POST /login': handleLogin,
11066     'POST /rank/submit': handleSubmit,
11067     'POST /rank/list': handleList,
11068     'POST /gameclub/daily': handleDailyPost,
11069 };

```

```
11070 exports.main = async (event, context) => {
11071   try {
11072     if (event && event.httpMethod === 'OPTIONS') {
11073       return preflight();
11074     }
11075     const req = parseEvent(event);
11076     const key = `${req.method} ${req.path}`;
11077     const handler = ROUTES[key];
11078     if (!handler) {
11079       return respond(404, { ok: false, code: 'NOT_FOUND', error: `no route: ${key}` });
11080     }
11081     const result = await handler(req, context);
11082     if (result && typeof result === 'object' && 'statusCode' in result) {
11083       return result;
11084     }
11085     return respond(200, { ok: true, data: result });
11086   } catch (error) {
11087     const code = error && error.code ? error.code : 'INTERNAL';
11088     const status = error && error.status ? error.status : 500;
11089     const message = (error && error.message) || String(error);
11090     console.error('[dresssort-api] error:', code, message, error && error.stack);
11091     const out = { ok: false, code, error: message };
11092     if (error && error.data !== undefined) {
11093       out.data = error.data;
11094     }
11095     return respond(status, out);
11096   }
11097 };
11098 // ===== FILE: cloudfunctions/dresssort-api/lib/auth.js =====
11099 const jwt = require('jsonwebtoken');
11100 const { httpError } = require('./http');
11101 const {
11102   getGameKey,
11103   gameKeyUpper,
11104   getJwtSecret: readJwtSecret,
11105   getTtlSec,
11106   getPlatformCredential,
11107 } = require('./config');
11108 const SUPPORTED_PLATFORMS = new Set(['wx', 'dy', 'tap', 'anon']);
11109 function getJwtSecret() {
11110   const secret = readJwtSecret();
11111   if (!secret) {
11112     throw httpError(500, 'NO_JWT_SECRET', `${gameKeyUpper()}_JWT_SECRET 未配置`);
11113   }
11114   return secret;
11115 }
11116 async function handleLogin(req) {
11117   const body = req.body || {};
11118   const platform = String(body.platform || '').toLowerCase();
11119   if (!SUPPORTED_PLATFORMS.has(platform)) {
```

```
11120   throw httpError(400, 'BAD_PLATFORM', `unsupported platform: ${platform}`);
11121 }
11122 let platformUid = "";
11123 let sessionKey = "";
11124 if (platform === 'wx') {
11125   const wx = await wxCode2Openid(body.code);
11126   platformUid = wx.openid;
11127   sessionKey = wx.sessionKey;
11128 } else if (platform === 'dy') {
11129   platformUid = await ttCode2Openid(body.code);
11130 } else if (platform === 'tap') {
11131   const id = String(body.taptapUserId || "").trim();
11132   if (!id) throw httpError(400, 'NO_TAP_ID', 'taptapUserId 缺失');
11133   platformUid = id;
11134 } else if (platform === 'anon') {
11135   const id = String(body.anonId || "").trim();
11136   if (!id) throw httpError(400, 'NO_ANON_ID', 'anonId 缺失');
11137   if (!/^[A-Za-z0-9_\-.]{8,128}$/.test(id)) {
11138     throw httpError(400, 'BAD_ANON_ID', 'anonId 非法');
11139   }
11140   platformUid = id;
11141 }
11142 const userId = `${platform}:${platformUid}`;
11143 if (sessionKey) {
11144   // 游戏圈数据是用 session_key 加密的，解密接口要用到。存失败不影响登录。
11145   try {
11146     await require('./game-club').upsertWxSession(userId, sessionKey);
11147   } catch (error) {
11148     console.warn('[auth] save wx session failed', error && error.message);
11149   }
11150 }
11151 const ttlSec = getTtlSec();
11152 const gameKey = getGameKey();
11153 const now = Math.floor(Date.now() / 1000);
11154 const token = jwt.sign(
11155   { sub: userId, plt: platform, gk: gameKey, iat: now },
11156   getJwtSecret(),
11157   { expiresIn: ttlSec },
11158 );
11159 return {
11160   token,
11161   userId,
11162   platform,
11163   gameKey,
11164   expiresAt: (now + ttlSec) * 1000,
11165   ttlSec,
11166 };
11167 }
11168 function requireUser(req) {
11169   const authHeader = (req.headers && req.headers.authorization) || "";
```

```
11170 const match = /^Bearer\s+(.+)$/i.exec(authHeader);
11171 if (!match) throw httpError(401, 'NO_TOKEN', '缺少 Authorization: Bearer <token>');
11172 let payload;
11173 try {
11174   payload = jwt.verify(match[1].trim(), getJwtSecret());
11175 } catch (error) {
11176   throw httpError(401, 'BAD_TOKEN', error && error.message ? error.message : 'token 无效');
11177 }
11178 const userId = payload && payload.sub;
11179 if (!userId || typeof userId !== 'string' || !userId.includes(':')) {
11180   throw httpError(401, 'BAD_TOKEN', 'token sub 非法');
11181 }
11182 const currentGameKey = getGameKey();
11183 if (payload.gk && payload.gk !== currentGameKey) {
11184   throw httpError(401, 'BAD_TOKEN', `token gameKey=${payload.gk} 与当前 GAME_KEY=${currentGameKey}...`);
11185 }
11186 return { userId, platform: payload.plt || userId.split(':')[0] };
11187 }
11188 async function wxCode2Openid(code) {
11189   const appid = getPlatformCredential('wx', 'APPID');
11190   const secret = getPlatformCredential('wx', 'SECRET');
11191   if (!appid || !secret) throw httpError(500, 'NO_WX_CFG', `${gameKeyUpper()}_WX_APPID/${gameKeyU...`);
11192   if (!code) throw httpError(400, 'NO_CODE', 'wx code 缺失');
11193   const url = `https://api.weixin.qq.com/sns/jscode2session?appid=${encodeURIComponent(appid)}&se...`;
11194   const data = await httpGetJson(url);
11195   if (!data || !data.openid) {
11196     throw httpError(401, 'WX_LOGIN_FAIL', `wx code2session 失败: ${JSON.stringify(data || {})} `);
11197   }
11198   return { openid: data.openid, sessionKey: data.session_key || '' };
11199 }
11200 async function ttCode2Openid(code) {
11201   const appid = getPlatformCredential('tt', 'APPID');
11202   const secret = getPlatformCredential('tt', 'SECRET');
11203   if (!appid || !secret) throw httpError(500, 'NO_TT_CFG', `${gameKeyUpper()}_TT_APPID/${gameKeyU...`);
11204   if (!code) throw httpError(400, 'NO_CODE', 'dy code 缺失');
11205   const data = await httpPostJson('https://developer.toutiao.com/api/apps/v2/jscode2session', { a...`);
11206   if (!data || data.err_no !== 0 || !data.data || !data.data.openid) {
11207     throw httpError(401, 'TT_LOGIN_FAIL', `dy code2session 失败: ${JSON.stringify(data || {})} `);
11208   }
11209   return data.data.openid;
11210 }
11211 function httpGetJson(url) {
11212   return httpRequestJson(url, 'GET');
11213 }
11214 function httpPostJson(url, body) {
11215   return httpRequestJson(url, 'POST', body);
11216 }
11217 function httpRequestJson(url, method, body) {
11218   if (typeof fetch !== 'function') {
11219     return Promise.reject(httpError(500, 'NO_FETCH', '当前 Node 运行时不支持 fetch, 请使用 Node 18+'));
```

```
11220 }
11221 const opts = { method, headers: { 'Content-Type': 'application/json' } };
11222 if (body !== undefined) {
11223   opts.body = JSON.stringify(body);
11224 }
11225 return fetch(url, opts).then(async (res) => {
11226   const text = await res.text();
11227   try {
11228     return JSON.parse(text);
11229   } catch (_) {
11230     return { _raw: text };
11231   }
11232 });
11233 }
11234 module.exports = {
11235   handleLogin,
11236   requireUser,
11237 };
11238 // ===== FILE: cloudfunctions/dresssort-api/lib/config.js =====
11239 const DEFAULT_GAME_KEY = 'dressort';
11240 const DEFAULT_TTL_SEC = 7 * 24 * 3600;
11241 const DEFAULT_MAX_BYTES = 256 * 1024;
11242 function getGameKey() {
11243   const v = String(process.env.GAME_KEY || '').trim().toLowerCase();
11244   if (!v) return DEFAULT_GAME_KEY;
11245   if (!/^[a-z][a-z0-9_-]{0,31}$/.test(v)) {
11246     throw new Error(`非法 GAME_KEY: ${v}`);
11247   }
11248   return v;
11249 }
11250 function gameKeyUpper() {
11251   return getGameKey().toUpperCase().replace(/[^A-Z0-9]/g, '_');
11252 }
11253 function readEnvPrefer(...keys) {
11254   for (const key of keys) {
11255     const value = process.env[key];
11256     if (value !== undefined && value !== null && String(value).length > 0) {
11257       return String(value);
11258     }
11259   }
11260   return '';
11261 }
11262 function getCollectionName(suffix) {
11263   const overrideKey = `${gameKeyUpper()}_${suffix.toUpperCase()}_COLLECTION`;
11264   const override = process.env[overrideKey];
11265   if (override) {
11266     return String(override);
11267   }
11268   return `${getGameKey()}_${suffix}`;
11269 }
```

```
11270 function getJwtSecret() {
11271   return readEnvPrefer(`${gameKeyUpper()}_JWT_SECRET`);
11272 }
11273 function getTtlSec() {
11274   const raw = readEnvPrefer(`${gameKeyUpper()}_TOKEN_TTL_SEC`);
11275   const v = Number(raw);
11276   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_TTL_SEC;
11277 }
11278 function getMaxBytes() {
11279   const raw = readEnvPrefer(`${gameKeyUpper()}_SAVE_MAX_BYTES`);
11280   const v = Number(raw);
11281   return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_MAX_BYTES;
11282 }
11283 function getPlatformCredential(platform, field) {
11284   const upper = gameKeyUpper();
11285   const platformUpper = platform.toUpperCase();
11286   return readEnvPrefer(`${upper}_${platformUpper}_${field}`);
11287 }
11288 module.exports = {
11289   getGameKey,
11290   gameKeyUpper,
11291   readEnvPrefer,
11292   getCollectionName,
11293   getJwtSecret,
11294   getTtlSec,
11295   getMaxBytes,
11296   getPlatformCredential,
11297 };
11298 // ===== FILE: cloudfunctions/dresssort-api/lib/db.js =====
11299 const tcb = require('@cloudbase/node-sdk');
11300 const { getCollectionName } = require('./config');
11301 let app = null;
11302 function getApp() {
11303   if (app) return app;
11304   app = tcb.init({
11305     env: process.env.TCB_ENV || tcb.SYMBOL_CURRENT_ENV,
11306   });
11307   return app;
11308 }
11309 function getDb() {
11310   return getApp().database();
11311 }
11312 function getCollection() {
11313   return getDb().collection(getCollectionName('playerData'));
11314 }
11315 function getRankingCollection() {
11316   return getDb().collection(getCollectionName('rankings'));
11317 }
11318 module.exports = {
11319   getApp,
```

```
11320   getDb,
11321   getCollection,
11322   getRankingCollection,
11323 };
11324 // ===== FILE: cloudfunctions/dresssort-api/lib/game-club.js =====
11325 /**
11326  * 游戏圈每日发帖任务。
11327  *
11328  * 客户端 wx.getGameClubData 拿到的是用 session_key 加密的数据,
11329  * 登录时把 session_key 存进 wxSessions, 这里解密后只回当天发帖数。
11330  * 奖励在客户端发 (跟着存档走), 服务端只负责证明「今天确实发过」。
11331  *
11332  * 集合: wxSessions (userId 唯一索引)
11333  */
11334 const crypto = require('crypto');
11335 const { httpError } = require('./http');
11336 const { getDb } = require('./db');
11337 const { getCollectionName } = require('./config');
11338 // wx.getGameClubData 的 dataType: 6 = 当天在本游戏圈发表的动态数。
11339 const DAILY_POST_TYPE = 6;
11340 function getSessionCollection() {
11341   return getDb().collection(getCollectionName('wxSessions'));
11342 }
11343 async function upsertWxSession(userId, sessionKey) {
11344   if (!sessionKey) return;
11345   const col = getSessionCollection();
11346   const now = Date.now();
11347   const res = await col.where({ userId }).limit(1).get();
11348   const existing = (res && Array.isArray(res.data) && res.data[0]) || null;
11349   if (existing && existing._id) {
11350     await col.doc(existing._id).update({ sessionKey, updatedAt: now });
11351     return;
11352   }
11353   await col.add({ userId, sessionKey, updatedAt: now });
11354 }
11355 async function readWxSessionKey(userId) {
11356   const res = await getSessionCollection().where({ userId }).limit(1).get();
11357   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
11358   return doc && typeof doc.sessionKey === 'string' ? doc.sessionKey : '';
11359 }
11360 function decrypt(sessionKey, encryptedData, iv) {
11361   try {
11362     const decipher = crypto.createDecipheriv(
11363       'aes-128-cbc',
11364       Buffer.from(sessionKey, 'base64'),
11365       Buffer.from(iv, 'base64'),
11366     );
11367     decipher.setAutoPadding(true);
11368     const text = Buffer.concat([
11369       decipher.update(Buffer.from(encryptedData, 'base64')),
```

```

11370     decipher.final(),
11371   ]).toString('utf8');
11372   return JSON.parse(text);
11373 } catch (error) {
11374   throw httpError(400, 'DECRYPT_FAIL', (error && error.message) || '游戏圈数据解密失败');
11375 }
11376 }
11377 function typeOf(entry) {
11378   const t = entry && entry.dataType;
11379   return typeof t === 'number' ? t : Number((t && t.type) || 0);
11380 }
11381 async function handleDailyPost(req) {
11382   // auth.js 登录时会反过来调这里存 session，所以懒加载避免循环依赖。
11383   const { requireUser } = require('./auth');
11384   const { userId, platform } = requireUser(req);
11385   if (platform !== 'wx') throw httpError(400, 'NOT_WX', '游戏圈只在微信里可用');
11386   const body = req.body || {};
11387   const encryptedData = String(body.encryptedData || '').trim();
11388   const iv = String(body.iv || '').trim();
11389   if (!encryptedData || !iv) throw httpError(400, 'BAD_GAME_CLUB_PAYLOAD', 'encryptedData / iv 缺失');
11390   const sessionKey = await readWxSessionKey(userId);
11391   if (!sessionKey) throw httpError(400, 'NO_WX_SESSION', '微信 session 未就绪，请重新登录');
11392   const data = decrypt(sessionKey, encryptedData, iv);
11393   const list = Array.isArray(data && data.dataList) ? data.dataList : [];
11394   const item = list.find((entry) => typeOf(entry) === DAILY_POST_TYPE);
11395   const n = Number(item && item.value);
11396   return { postCount: Number.isFinite(n) && n > 0 ? Math.floor(n) : 0 };
11397 }
11398 module.exports = {
11399   DAILY_POST_TYPE,
11400   upsertWxSession,
11401   handleDailyPost,
11402 };
11403 // ===== FILE: cloudfunctions/dresssort-api/lib/http.js =====
11404 const { getGameKey } = require('./config');
11405 const CORS_HEADERS = {
11406   'Access-Control-Allow-Origin': '*',
11407   'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
11408   'Access-Control-Allow-Headers': 'Content-Type,Authorization',
11409   'Access-Control-Max-Age': '86400',
11410 };
11411 function respond(statusCode, body, extraHeaders = {}) {
11412   return {
11413     statusCode,
11414     headers: {
11415       'Content-Type': 'application/json; charset=utf-8',
11416       ...CORS_HEADERS,
11417       ...extraHeaders,
11418     },
11419     body: typeof body === 'string' ? body : JSON.stringify(body),

```

```
11420     isBase64Encoded: false,
11421   };
11422 }
11423 function preflight() {
11424   return {
11425     statusCode: 204,
11426     headers: { ...CORS_HEADERS },
11427     body: "",
11428     isBase64Encoded: false,
11429   };
11430 }
11431 function parseEvent(event) {
11432   event = event || {};
11433   if (event.httpMethod) {
11434     const method = String(event.httpMethod).toUpperCase();
11435     let rawBody = event.body || "";
11436     if (event.isBase64Encoded && rawBody) {
11437       try {
11438         rawBody = Buffer.from(rawBody, 'base64').toString('utf8');
11439       } catch (_) {}
11440     }
11441     let body = {};
11442     if (rawBody) {
11443       try {
11444         body = JSON.parse(rawBody);
11445       } catch (_) {
11446         body = {};
11447       }
11448     }
11449     return {
11450       method,
11451       path: normalizePath(event.path || '/'),
11452       body,
11453       headers: lowercaseHeaders(event.headers || {}),
11454       query: event.queryStringParameters || {},
11455       raw: event,
11456     };
11457   }
11458   const action = String(event.action || "").replace(/^\/+/, "");
11459   return {
11460     method: 'POST',
11461     path: action ? `/${action}` : '/',
11462     body: event.body || {},
11463     headers: lowercaseHeaders(event.headers || {}),
11464     query: {},
11465     raw: event,
11466   };
11467 }
11468 function normalizePath(path) {
11469   if (!path) return '/';
```

```
11470 let p = String(path);
11471 if (!p.startsWith('/')) p = `/${p}`;
11472 const apiPrefix = `${getGameKey()}-api`;
11473 p = p.replace(new RegExp(`^(?:${escapeRegExp(apiPrefix)})(?=/|$)`), "");
11474 if (p === "") p = '/';
11475 if (p.length > 1 && p.endsWith('/')) p = p.slice(0, -1);
11476 return p;
11477 }
11478 function lowercaseHeaders(headers) {
11479   const out = {};
11480   for (const key of Object.keys(headers | {})) {
11481     out[key.toLowerCase()] = headers[key];
11482   }
11483   return out;
11484 }
11485 function httpError(status, code, message) {
11486   const err = new Error(message || code);
11487   err.status = status;
11488   err.code = code;
11489   return err;
11490 }
11491 function escapeRegExp(text) {
11492   return String(text).replace(/[*+?^$*{}()|[\]\\"/g, '\\$&');
11493 }
11494 module.exports = {
11495   respond,
11496   preflight,
11497   parseEvent,
11498   httpError,
11499 };
11500 // ===== FILE: cloudfunctions/dresssort-api/lib/rank.js =====
11501 /**
11502  * 通关排行榜，只有一张榜：已通关关数。
11503  *
11504  * 数据模型（collection: rankings，每人一条）：
11505  * {
11506  *   _id,
11507  *   userId, platform,
11508  *   cleared: number,    // 已通关关数
11509  *   reachedAt: number (ms), // 第一次到达这个关数的时间，同分先到者排前面
11510  *   displayName, avatarUrl,
11511  *   createdAt, updatedAt
11512  * }
11513  *
11514  * 索引：userId 唯一； cleared desc + reachedAt asc。
11515  */
11516 const crypto = require('crypto');
11517 const { requireUser } = require('./auth');
11518 const { getDb, getRankingCollection } = require('./db');
11519 const { httpError } = require('./http');
```

```
11520 const { readEnvPrefer, gameKeyUpper } = require('./config');
11521 const DEFAULT_MAX_CLEARED = 500;
11522 const DEFAULT_LIST_LIMIT = 50;
11523 const MAX_LIST_LIMIT = 100;
11524 function envInt(suffix, fallback) {
11525   const n = Number(readEnvPrefer(`${gameKeyUpper()}_${suffix}`));
11526   return Number.isFinite(n) && n > 0 ? Math.floor(n) : fallback;
11527 }
11528 // GM / 测试账号: 逗号分隔的 userId, 不写榜, 免得一键全通关的进度挂在榜首。
11529 function blockedUsers() {
11530   return new Set(
11531     readEnvPrefer(`${gameKeyUpper()}_RANK_BLOCKED_UIDS`)
11532       .split(',')
11533       .map((s) => s.trim())
11534       .filter(Boolean),
11535   );
11536 }
11537 function normalizeCleared(value) {
11538   const n = Number(value);
11539   if (!Number.isFinite(n) || n < 0) {
11540     throw httpError(400, 'BAD_CLEARED', 'cleared 必须为非负整数');
11541   }
11542   const cleared = Math.floor(n);
11543   const max = envInt('RANK_MAX_CLEARED', DEFAULT_MAX_CLEARED);
11544   if (cleared > max) {
11545     throw httpError(400, 'CLEARED_TOO_HIGH', `cleared 超出上限: ${cleared} > ${max}`);
11546   }
11547   return cleared;
11548 }
11549 function normalizeLimit(value) {
11550   const n = Number(value);
11551   if (!Number.isFinite(n) || n <= 0) return DEFAULT_LIST_LIMIT;
11552   return Math.min(Math.floor(n), MAX_LIST_LIMIT);
11553 }
11554 function displayNameForUser(userId) {
11555   const digest = crypto.createHash('sha1').update(String(userId)).digest('hex');
11556   const suffix = String(parseInt(digest.slice(0, 8), 16) % 10000).padStart(4, '0');
11557   return `裙友${suffix}`;
11558 }
11559 function sanitizeDisplayName(value, fallback) {
11560   const text = String(value || '').trim().replace(/[\r\n\t]/g, ' ').slice(0, 16);
11561   // 微信拒绝授权时会给一个统一的「微信用户」, 当没给。
11562   if (!text || text === '微信用户') return fallback;
11563   return text;
11564 }
11565 function sanitizeAvatarUrl(value, fallback) {
11566   const text = String(value || '').trim();
11567   if (!text || text.length > 1024 || !/^https?:\/\/\./i.test(text)) return fallback || '';
11568   return text;
11569 }
```

```
11570 function publicRecord(doc, userId, rank) {
11571   if (!doc) return null;
11572   return {
11573     rank: rank || 0,
11574     cleared: Number(doc.cleared) || 0,
11575     displayName: doc.displayName || displayNameForUser(doc.userId || ''),
11576     avatarUrl: doc.avatarUrl || '',
11577     isMe: !!userId && doc.userId === userId,
11578   };
11579 }
11580 // 编辑器用 anon 身份联调。微信玩家看不到 anon 记录，编辑器里能看到全部，
11581 // 这样本地调界面有数据可看，线上榜也不会混进测试号。
11582 function audience(platform) {
11583   return platform === 'anon' ? {} : { platform: getDb().command.neq('anon') };
11584 }
11585 async function findMine(col, userId, platform) {
11586   const res = await col.where({ userId }).limit(1).get();
11587   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
11588   if (!doc || !(Number(doc.cleared) > 0)) return null;
11589   const _ = getDb().command;
11590   const cleared = Number(doc.cleared);
11591   const reachedAt = Number(doc.reachedAt) || 0;
11592   const scope = audience(platform);
11593   const [ahead, tied] = await Promise.all([
11594     col.where({ ...scope, cleared: _.gt(cleared) }).count(),
11595     col.where({ ...scope, cleared, reachedAt: _.lt(reachedAt) }).count(),
11596   ]);
11597   const rank = (Number(ahead && ahead.total) || 0) + (Number(tied && tied.total) || 0) + 1;
11598   return { rank, doc };
11599 }
11600 async function handleSubmit(req) {
11601   const { userId, platform } = requireUser(req);
11602   const body = req.body || {};
11603   const cleared = normalizeCleared(body.cleared);
11604   if (blockedUsers().has(userId)) {
11605     return { updated: false, reason: 'BLOCKED', record: null };
11606   }
11607   const col = getRankingCollection();
11608   const res = await col.where({ userId }).limit(1).get();
11609   const existing = (res && Array.isArray(res.data) && res.data[0]) || null;
11610   const now = Date.now();
11611   const displayName = sanitizeDisplayName(
11612     body.displayName,
11613     (existing && existing.displayName) || displayNameForUser(userId),
11614   );
11615   const avatarUrl = sanitizeAvatarUrl(body.avatarUrl, (existing && existing.avatarUrl) || '');
11616   if (!existing) {
11617     if (cleared <= 0) return { updated: false, reason: 'NO_PROGRESS', record: null };
11618     const docData = {
11619       userId,
```

```
11620   platform,
11621   cleared,
11622   reachedAt: now,
11623   displayName,
11624   avatarUrl,
11625   createdAt: now,
11626   updatedAt: now,
11627   };
11628   await col.add(docData);
11629   return { updated: true, mode: 'insert', record: publicRecord(docData, userId, 0) };
11630 }
11631 const prev = Number(existing.cleared) || 0;
11632 const patch = {};
11633 // 只升不降：清档重玩、换设备拿到旧档都不会把榜上的成绩拉下来。
11634 if (cleared > prev) {
11635   patch.cleared = cleared;
11636   patch.reachedAt = now;
11637 }
11638 if (existing.displayName !== displayName) patch.displayName = displayName;
11639 if ((existing.avatarUrl || '') !== avatarUrl) patch.avatarUrl = avatarUrl;
11640 if (Object.keys(patch).length === 0) {
11641   return { updated: false, reason: 'NOT_BETTER', record: publicRecord(existing, userId, 0) };
11642 }
11643 patch.updatedAt = now;
11644 await col.doc(existing._id).update(patch);
11645 return {
11646   updated: true,
11647   mode: patch.cleared !== undefined ? 'update' : 'profile_update',
11648   record: publicRecord({ ...existing, ...patch }, userId, 0),
11649 };
11650 }
11651 async function handleList(req) {
11652   const { userId, platform } = requireUser(req);
11653   const limit = normalizeLimit((req.body || {}).limit);
11654   const col = getRankingCollection();
11655   const res = await col
11656     .where({ ...audience(platform), cleared: getDb().command.gt(0) })
11657     .orderBy('cleared', 'desc')
11658     .orderBy('reachedAt', 'asc')
11659     .limit(limit)
11660     .get();
11661   const rows = res && Array.isArray(res.data) ? res.data : [];
11662   const list = rows.map((doc, i) => publicRecord(doc, userId, i + 1));
11663   const mine = await findMine(col, userId, platform);
11664   return { list, mine: mine ? publicRecord(mine.doc, userId, mine.rank) : null };
11665 }
11666 module.exports = {
11667   handleSubmit,
11668   handleList,
11669 };
```